

Modified Adaptative Blending Units Residual for Physics-Informed Neuronal Network

Sierra Salamanca Julián

11 de junio de 2026

Resumen

En este trabajo se propone y evalúa una arquitectura modificada M4 (Wang et al. 2020), integrando unidades adaptativas de activación (ABU, Wang et al., 2023), y conexiones residuales, con el objetivo de mejorar la representabilidad y expresividad de la red para EDPs hiperbólicas. Mediante un análisis teórico y experimentos numéricos, se compara el desempeño de la arquitectura propuesta frente a redes PINNs propuestas hasta el momento para estas EDPs: perceptron multicapa con función de activación tanh, perceptron multicapa con función de activación SIREN, MultiScalePINNs, modified MLP (M4) y ABU. Repositorio <https://github.com/Emmaquantum/HPINN>

1. Introducción

Las ecuaciones diferenciales parciales (EDP) hiperbólicas son fundamentales en la descripción de fenómenos de onda, como la propagación del sonido, la dinámica de fluidos y las vibraciones mecánicas. En particular, las EDPs hiperbólicas dispersivas, como la ecuación de Klein-Gordon, son ampliamente utilizadas en cosmología y física de altas energías (Yagdjian, 2013), en el estudio de solitones y física cuántica (Madani et al., 2025), en fenómenos dispersivos y física de fluidos (Miller et al., 2019), así como en materia condensada (Megías et al., 2022). Su solución numérica presenta desafíos significativos debido a la naturaleza oscilatoria y la conservación de la energía que las caracteriza.

En los últimos años, las redes neuronales informadas por la física (Physics-Informed Neural Networks, PINNs) se han consolidado como un marco prometedor para resolver problemas directos e inversos, es decir, hayar soluciones directas de EDPs o hayar coeficientes a partir de datos obtenidos (Raissi et al., 2019). No obstante, se ha demostrado que las PINNs convencionales sufren de patologías a la hora de entrenarse, especialmente, cuando se enfrentan a soluciones de alta frecuencia, múltiples escalas o regímenes de largo tiempo (Wang et al., 2020; Wang et al., 2021; Wang et al., 2022). Entre estas dificultades se encuentran el sesgo espectral (spectral bias), que impide aprender adecuadamente las componentes de alta frecuencia y baja frecuencia al mismo tiempo (Wang et al., 2021); el desbalance en los gradientes entre los términos involucrados en la función de optimización y la rigidez en la dinámica del flujo de gradiente (Wang et al., 2020); y la violación de la estructura de causalidad temporal inherente a los sistemas dinámicos, en especial a las EDPs de dispersión (Wang et al., 2022).

Para mitigar estas patologías, se han propuesto diversas mejoras arquitectónicas y empíricas. Por ejemplo, el uso de Fourier feature embeddings le entrega a la red una variedad de espectros del Neural Tangent Kernel (NTK), para así darle expresividad respecto a una variedad de frecuencias a la red (Wang et al., 2021; Jacot et al., 2018); esto mismo se logra a partir del uso de funciones de activación SIREN (Chrisnanto, 2026). Asimismo, estrategias de entrenamiento causal han demostrado mejorar la propagación del frente de onda y evitar mínimos óptimos que satisfacen la EDP pero no logran representabilidad física, es decir, engañan a la red (Wang et al., 2022). Por otro lado, la incorporación de funciones de activación adaptativas, como las Adaptive Blending Units (ABU) (Wang et al., 2023) ajustan la forma de la activación durante el entrenamiento, y esto permite mitigar la complejidad del paisaje de pérdida y ganar representabilidad; todo esto depende del conjunto de funciones de activación elegidas. Recientemente Wang et al. (2024) presentaron la arquitectura Physics-Informed Residual Adaptive Networks (PirateNets) en el que introduce conexiones residuales adaptativas que inicializan la red como una combinación lineal de características y profundizan progresivamente a comportamientos no lineales; esto permite resolver problemas de inicialización patológica y de representación que afectan a las redes profundas (Wang et al., 2024).

En este trabajo se propone y evalúa una arquitectura modificada (modified MLP) que integra las unidades adaptativas de activación (ABU) y conexiones residuales (Como las de PirateNet), como

una alternativa para mejorar la representabilidad y expresividad de la red frente a EDPs hiperbólicas. Se realiza un análisis teórico y experimental del desempeño de estas técnicas, centrándose en la solución de la ecuación de Klein-Gordon lineal homogénea. Los resultados obtenidos permiten identificar las ventajas y limitaciones de las arquitecturas actuales, abriendo nuevas posibilidades para el uso de PINNs en problemas de ondas dispersivas.

2. Marco teórico

2.1. Ecuación de Klein-Gordon

Se considera la ecuación de Klein-Gordon para el presente estudio por ser la EDP dispersiva con amortiguamiento, esto nos permite ajustar una serie de condiciones iniciales tales que presenten desafíos a la red como sesgo espectral y propagación. Matemáticamente, estamos hablando de una ecuación diferencial parcial hiperbólica para un campo escalar:

$$\underbrace{\frac{1}{c^2} \frac{\partial^2}{\partial t^2} \phi(\mathbf{x}, t)}_{\text{Evolución temporal}} - \underbrace{\nabla^2 \phi(\mathbf{x}, t)}_{\text{Difusión}} + \underbrace{\left(\frac{mc}{\hbar} \right)^2 \phi(\mathbf{x}, t)}_{\text{Fuerza restauradora}} = 0. \quad (1)$$

Dónde encontramos el término de segunda derivada temporal ∂_{tt} como una fuerza que induce el cambio y la dinámica del sistema, el término de difusión ∇^2 y un término de m^2 que actúa como una fuerza restauradora. La ecuación de Klein-Gordon se puede interpretar en una sola dimensión espacial como una onda que perturba una cuerda en un tiempo arbitrario.

Se quiere que la solución contenga las frecuencias $e^{\pm i\mathbf{k} \cdot \mathbf{x}}$. Esta solución se puede hallar mediante el método de separación de variables propuesto por Fourier:

$$\phi(\mathbf{x}, t) = \Psi(\mathbf{x})T(t). \quad (2)$$

Hallamos las derivadas temporales y espaciales:

$$\frac{\partial^2 \phi}{\partial t^2} = \Psi(\mathbf{x})T''(t) \quad (3)$$

$$\frac{\partial^2 \phi}{\partial x^2} = \Psi''(\mathbf{x})T(t). \quad (4)$$

Reemplazando esta solución a la ecuación diferencial:

$$\frac{1}{c^2} \Psi(\mathbf{x})T''(t) - \Psi''(\mathbf{x})T(t) + \left(\frac{mc}{\hbar} \right)^2 \Psi(\mathbf{x})T(t) = 0. \quad (5)$$

Separamos variables:

$$\frac{1}{c^2} \frac{T''(t)}{T(t)} + \mu^2 = \frac{\Psi''(\mathbf{x})}{\Psi(\mathbf{x})}$$

Dónde $\mu = mc/\hbar$. Para que esta identidad se cumpla cada término, el derecho y el izquierdo de la igualdad deben ser igual a una constante $-k^2$, de este modo, obtenemos ODEs para cada función por separado:

$$\frac{1}{c^2} T''(t) + i\omega_k^2 T(t) = 0, \quad \Psi''(\mathbf{x}) + k^2 \Psi(\mathbf{x}) = 0.$$

En dónde, $\omega_k^2 = (\mu^2 + k^2)$; ambas son ecuaciones diferenciales del armónico simple, por lo tanto sus soluciones son:

$$\Psi(\mathbf{x}) = Ae^{i\mathbf{k} \cdot \mathbf{x}} + Be^{-i\mathbf{k} \cdot \mathbf{x}}, \quad (6)$$

$$T(t) = Ce^{i\omega_k t} + De^{-i\omega_k t}. \quad (7)$$

Actualizando las soluciones de cada función se obtiene la solución general del campo de Klein-Gordon,

$$\begin{aligned} \phi(\mathbf{x}, t) &= \Psi(\mathbf{x})T(t) \\ &= (Ae^{i\mathbf{k} \cdot \mathbf{x}} + Be^{-i\mathbf{k} \cdot \mathbf{x}})(Ce^{i\omega_k t} + De^{-i\omega_k t}) \\ &= K_1 e^{i(\mathbf{k} \cdot \mathbf{x} - \omega_k t)} + K_2 e^{-i(\mathbf{k} \cdot \mathbf{x} - \omega_k t)}. \end{aligned} \quad (8)$$

Siendo $\mathbf{x} = (x, y, z)$ y $\mathbf{k} = (k_x, k_y, k_z)$. Este resultado se puede generalizar aún más para un electrón con una energía cuantizada en diferentes fases:

$$\phi(\mathbf{x}, t) = \sum_{\mathbf{k}} K_{1k} e^{i(\mathbf{k} \cdot \mathbf{x} - \omega_k t)} + K_{2k} e^{-i(\mathbf{k} \cdot \mathbf{x} - \omega_k t)}. \quad (9)$$

Para el caso simple, tenemos que la solución en una dimensión (1-D) es simplemente la solución:

$$\phi(x, t) = \sum_k K_{1k} e^{i(kx - \omega_k t)} + K_{2k} e^{-i(kx - \omega_k t)}. \quad (10)$$

2.2. Condiciones iniciales y de frontera

Las condiciones iniciales y de frontera serán de tal modo que representen una partícula estacionaria, es decir, con posición inicial y velocidad nula.

- **Condición inicial.** Las condiciones iniciales, tanto del campo y de la velocidad del campo, será modelada por un pulso Gaussiano y una función que induzca velocidad en $+k$ y $-k$:

$$\phi(\mathbf{x}, 0) = \underbrace{Ae^{-(x^2/2\sigma)}}_{\text{Gausiana}} \underbrace{\sum_{\mathbf{k}_0} (e^{i\mathbf{k}_0 \cdot \mathbf{x}} + e^{-i\mathbf{k}_0 \cdot \mathbf{x}})}_{\text{Fase simétrica}}, \quad (\text{Posición}) \quad (11)$$

$$\dot{\phi}(\mathbf{x}, 0) = 0. \quad (\text{Velocidad}) \quad (12)$$

Esta condición inicial nos permite manipularla concientemente para que la red tenga la necesidad de aprender altas y bajas frecuencias al introducirle un vector $\mathbf{k} = [k_1, k_2, \dots, k_n]$, en dónde $k_n > k_{n-1} > \dots > k_1$.

- **Condición de frontera.** En el método pseudoespectral la condición de frontera es una condición periódica.

$$\phi(-L/2, t) = \phi(L/2, t) \quad (13)$$

Sin embargo, en la simulación, la onda no se dispersará hasta la frontera debido a los parámetros establecidos de tiempo y espacio.

2.3. Solución exacta

Aplicando las condiciones iniciales para el caso en una dimensión (1-D), encontramos que $k_0 = k$:

$$\sum_k K_{1k} e^{ikx} + K_{2k} e^{-ikx} = Ae^{-(x^2/2\sigma)} \sum_k (e^{ikx} + e^{-ikx}) \quad (14)$$

$$\sum_k (K_{1k} - Ae^{-(x^2/2\sigma)}) e^{ikx} + (K_{2k} - Ae^{-(x^2/2\sigma)}) e^{-ikx} = 0 \quad (15)$$

Cada término en la sumatoria debe anularse; para el caso trivial $(K_{1k} - Ae^{-(x^2/2\sigma)}) = (K_{2k} - Ae^{-(x^2/2\sigma)}) = 0$ debe cumplirse, así que $K_{1k} = K_{2k} = Ae^{-(x^2/2\sigma)} = K_k$. Así que la solución con condición inicial es:

$$\phi(x, t) = \sum_k K_k (e^{i(\mathbf{k} \cdot \mathbf{x} - \omega_k t)} + e^{-i(\mathbf{k} \cdot \mathbf{x} - \omega_k t)}) \quad (16)$$

$$= \sum_k 2K_k \cos(kx - \omega_k t) \quad (17)$$

$$= \underbrace{\left(\sum_k 2K_k \cos(kx) \right)}_{\phi(x,0)} \cos(\omega_k t) + \underbrace{\left(\sum_k 2K_k \sin(kx) \right)}_{\frac{\dot{\phi}(x,0)}{\omega_k}} \sin(\omega_k t) \quad (18)$$

$$= \phi(x, 0) \cos(\omega_k t) + \frac{\dot{\phi}(x, 0)}{\omega_k} \sin(\omega_k t) \quad (19)$$

La solución exacta depende de la posición y velocidad iniciales, y a su vez de una base de fourier temporal. Al aplicar la condición de velocidad inicial $\dot{\phi}(x, 0) = 0$, nos quedamos con una solución relativamente simple:

$$\phi(x, t) = \phi(x, 0) \cos(\omega_k t) \quad (20)$$

Esto nos hace pensar que una función de activación SIREN o un *embedding Fourier Transform* bastaría con darle representatividad a la red.

3. Red Neuronal Informada por la física

Si bien las redes neuronales presentan una gran capacidad de aproximación a funciones, las arquitecturas profundas y altamente conectadas presentan dificultad para modelar soluciones de altas frecuencias y características multiescala (Wang et al., 2022). Según Wang et al. (2021), esto se atribuye a las interacciones conflictivas en la función de pérdida; sin embargo las causas fundamentales de este comportamiento aún no se comprenden totalmente.

Yu et al. (2022) propone el método de gPINN (gradient-enhanced PINN), el cual introduce un nuevo término en la función de pérdida basado en las derivadas del residual de la EDP; su argumento se basa en que si la EDP debe ser igual a cero en la solución, entonces su gradiente espacial también debe ser cero, anexando así restricciones que mejoran la precisión y la eficiencia del entrenamiento.

Wang et al. (2021) argumenta que el problema de usar un algoritmo como ADAM está en la rigidez en la dinámica del flujo de gradiente, esto quiere decir, las magnitudes de los gradientes que proviene de la física ($\nabla_{\theta} \mathcal{L}_r$) suele ser más grandes que los gradientes que provienen de las condiciones iniciales y de contorno ($\nabla_{\theta} \mathcal{L}_i$ donde $i = \{ic, bc\}$); debido a que ADAM intenta minimizar el error global, dedicará mucho tiempo en minimizar el error del que tenga mayor gradiente, es decir, el gradiente que gobierne la función de costo.

3.1. Definición de Red Neuronal Informada por la Física - PINN

La PINN es un método de machine learning que usa redes neuronales (perceptrones multicapas) para aproximar la solución de una EDP a través del conocimiento específico del dominio, la física y las condiciones iniciales. Partimos de la definición de la EDP:

Definición 3.1: EDP hiperbólica generica

Según Qian et al. (2023), una EDP de segundo orden en el tiempo, se considera un dominio compacto $x \in D \subseteq \mathcal{R}^n$ y $t \in [0, T]$. Con los operadores diferencial y de contorno $\mathcal{D}$ y $\mathcal{B}$. Consideramos la forma más general de la EDP:

$$\begin{aligned} \frac{\partial^2 u}{\partial t^2}(\mathbf{x}, t) + \mathcal{D}[u](\mathbf{x}, t) &= f \\ \mathcal{B}[u](\mathbf{x}, t) &= g \\ u(\mathbf{x}, 0) &= h \\ \dot{u}(\mathbf{x}, 0) &= v \end{aligned}$$

Una PINN es una aproximador $\phi(\mathbf{x}, t; \boldsymbol{\theta}) = NN(\mathbf{x}, t)$ a la función $u(\mathbf{x}, t)$ solución de la EDP. De este modo, definimos el residuo como:

$$\frac{\partial^2 u}{\partial t^2}(\mathbf{x}, t) + \mathcal{D}[u](\mathbf{x}, t) = \underbrace{r(\mathbf{x}, t, \boldsymbol{\theta})}_{\text{Residuo de la física}} = f. \quad (21)$$

3.2. Función Costo y convergencia

Una buena aproximación es aquella que el residuo $r(\mathbf{x}, t, \boldsymbol{\theta})$ sea lo más cercano a cero. Se implementa una función de costo para la optimización de los parámetros capaces de minimizar el residuo físico.

$$\mathcal{L}(\boldsymbol{\theta}) = \mathcal{L}_{data}(\boldsymbol{\theta}) + \mathcal{L}_f(\boldsymbol{\theta}), \quad (22)$$

$$= \mathcal{L}_{bc}(\boldsymbol{\theta}, \mathcal{T}_{bc}) + \mathcal{L}_{bi}(\boldsymbol{\theta}, \mathcal{T}_{bi}) + \mathcal{L}_f(\boldsymbol{\theta}, \mathcal{T}_f). \quad (23)$$

En donde,

$$\mathcal{L}_f(\boldsymbol{\theta}, \mathcal{T}_f) = \frac{1}{|\mathcal{T}_f|} \sum_{\mathbf{x} \in \mathcal{T}_f} |r(\mathbf{x}, t, \boldsymbol{\theta})|^2, \quad (24)$$

$$\mathcal{L}_{ic}(\boldsymbol{\theta}, \mathcal{T}_{ic}) = \frac{1}{|\mathcal{T}_{ic}|} \sum_{\mathbf{x} \in \mathcal{T}_{ic}} |\phi(\mathbf{x}, 0, \boldsymbol{\theta}) - h(\mathbf{x}, 0)|^2, \quad (25)$$

$$\mathcal{L}_{bc}(\boldsymbol{\theta}, \mathcal{T}_{bc}) = \frac{1}{|\mathcal{T}_{bc}|} \sum_{\mathbf{x} \in \mathcal{T}_{bc}} |\phi(L, t, \boldsymbol{\theta}) - g(\mathbf{x}, t)|^2. \quad (26)$$

Cuando se tiene datos experimentales, se debe agregar a la función de costo asociada a los datos. Se definen tres tipos de errores: error de optimización, error de aproximación y error de estimación.

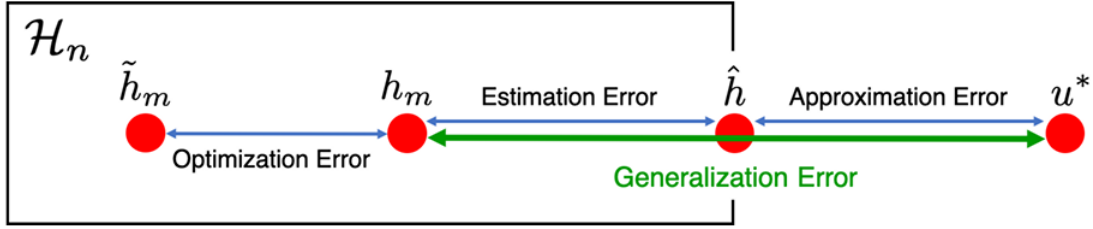


Figura 1: Diagrama de diferentes errores que se pueden presentar al entrenar una PINN. Sacado de Shin et al., 2020.

El error de optimización es la diferencia entre la red que tengo entrenada con la mejor versión de la red entrenada hasta ahora. El error de aproximación es la diferencia entre la función exacta (o el método numérico estandar) con la solución de mi red. El error de estimación es el error asociado a puntos del dominio que no fueron parte del entrenamiento y comparados con la solución exacta. Cuando se habla de estos tres tipos de red, se asume que existe una red que es mi aproximación con respecto a una red esperada (teórica o empíricamente mejor entrenada).

Es natural intentar acotar la solución teórica con respecto a la solución que obtengo a partir de ciertas iteraciones m . Para esto, se contruye la función de pérdida (Costo) de la red de tal manera que se intente penalizar el crecimiento de alguno de estos términos involucrados;

$$Loss_m(u; \lambda, \lambda_m^R) = Loss_m^{PINN}(h; \lambda) + \hat{\lambda}_f^R [\mathcal{L}_f[u]]_{\alpha, U}^2 + \hat{\lambda}_{bc}^R [\mathcal{L}_{bc}[u]]_{\alpha, U}^2 + \hat{\lambda}_{ic}^R [\mathcal{L}_{ic}[u]]_{\alpha, U}^2 \quad (27)$$

En dónde,

$$[\mathcal{L}_i[h]]_{\alpha; U} = \sup_{\substack{\mathbf{x}, \mathbf{y} \in U \\ \mathbf{x} \neq \mathbf{y}}} \frac{\|\mathcal{L}_i[h](\mathbf{x}) - \mathcal{L}_i[h](\mathbf{y})\|}{\|\mathbf{x} - \mathbf{y}\|^\alpha}, \quad 0 < \alpha \leq 1. \quad (28)$$

Se asume que son funciones continuas de Hölder por motivos de no restricción, es decir, se puede asumir funciones continuas de Lipschitz (funciones suaves y continuas) pero esto llevaría a casos específicos y asunciones fuertes de lo que sucede, por tal motivo, se generaliza a funciones continuas de Hölder (Shin et al., 2020).

$$Loss^{PINN}(h; \lambda) \leq C_m \cdot Loss_m(h; \lambda, \hat{\lambda}_m^R) + C'(m_r^{-\alpha/n} + m_b^{-\alpha/(n-1)}) \quad (29)$$

Demostrango que controlar la función de pérdida empírica $Loss_m(h; \lambda, \hat{\lambda}_m^R)$ regularizada controla la función de pérdida esperada $Loss^{PINN}(h; \lambda)$. Este enfoque es generalizable siempre y cuando supongamos que $\mathcal{L}_i[u]$, f , g son funciones continuas de Hölder (Shin et al., 2020).

Las EDPs parabólicas y elípticas, bajo hipótesis de regularización de Hölder y regularización adecuada como se acaba de mostrar, permiten demostrar que la sucesión de minimizadores u_m convergen a la solución clásica u^* en la norma uniforme $C^0(U)$ para parabólicas y $C^0(\bar{U}_T)$ para elípticas. Es decir,

$$\lim_{m \rightarrow \infty} \|h_m - u^*\| = 0 \quad (30)$$

Esto se obtiene gracias a que la cota obtenida en la ecuación (29), la equicontinuidad uniforme de $\{\mathcal{B}[u_m]\}$ y $\{\mathcal{L}[u_m]\}$ inducida por la regularización de Hölder, y por ultimo, el principio del máximo permiten la convergencia de las PINNs hacia la solución esperada.

En cambio, las EDPs hiperbólicas tiene particularidades en sus soluciones como no presentar un máximo en su solución. Las soluciones de las hiperbólicas presentan periodicidad y ondas viajeras lo cuál no dan paso a una convergencia uniforme a partir del control del residuo (Qian et al., 2023).

Una alternativa natural es trabajar a partir de la conservación de energía que satisfacen este tipo de ecuaciones, al ser ecuaciones dispersivas, la energía debe conservarse. Esto permite hacer análisis de conservación de energía en espacios de Sobolev, y por lo tanto ampliar un poco la ecuación (27) agregandole un término de regularización de energía. Qian et al. (2023) realiza este análisis a partir de una reformulación de la función de pérdida que incorpora residuos adicionales: el gradiente del residuo interior, el gradiente del residuo de la condición inicial, y el uso de la raíz cuadrada en las condiciones de contorno. Estos términos, ausentes en la PINN estándar, permiten acotar el error en normas de Sobolev y probar la convergencia de la aproximación para ecuaciones como la de onda, Sine-Gordon y elastodinámica lineal, sin necesidad de añadir regularización artificial de energía.

3.3. Problema de optimización

3.3.1. Minimización con restricción de igualdad (restricción dura)

Entonces, el problema de optimización recae en minimizar la función de pérdida $\mathcal{L} = \mathcal{L}_{ic} + \mathcal{L}_{bc}$ con respecto a los parámetros θ sujeto a la función de pérdida física $\mathcal{L}_f$ (Krishnapriyan et al., 2021):

$$\min_{\theta} \quad \mathcal{L}_{ic}(\theta) + \mathcal{L}_{bc}(\theta), \quad (31)$$

$$\text{s.a.} \quad \mathcal{L}_f(\theta) = 0. \quad (32)$$

Según Krishnapriyan et al. (2021), esta aproximación es costosa de resolver.

3.3.1.1. Lagrangiano y condiciones KKT

La función Lagrangiana introduce el vector dual $\lambda \in \mathbb{R}^N$,

$$\mathcal{L}(\theta, \mathbf{v}) = \mathcal{L}(\theta)_{data} + \mathbf{v}^T h(\theta), \quad (33)$$

$$= (\mathcal{L}_{ic}(\theta) + \mathcal{L}_{bc}(\theta)) + v\mathcal{L}_f(\theta). \quad (34)$$

Las condiciones KKT, una vez calculado el Lagrangiano, son:

$$\nabla_{\theta} \mathcal{L}_{ic}(\theta^*) + \nabla_{\theta} \mathcal{L}_{bc}(\theta^*) + v \nabla_{\theta} \mathcal{L}_f(\theta^*) = 0, \quad (35)$$

$$\mathcal{L}_f(\theta^*) = 0. \quad (36)$$

Al desarrollar el gradiente de la condición inicial,

$$\nabla_{\theta} \mathcal{L}_{ic}(\theta^*) = \nabla_{\theta} \left(\frac{1}{|\mathcal{T}_{ic}|} \sum_{x_{ic} \in \mathcal{T}_{ic}} |\phi(x, 0, \theta) - h(x, 0)|^2 \right), \quad (37)$$

$$= \frac{1}{|\mathcal{T}_{ic}|} \sum_{x \in \mathcal{T}_{ic}} \nabla_{\theta} |\phi(x, 0, \theta) - h(x, 0)|^2, \quad (38)$$

$$= \frac{2}{|\mathcal{T}_{ic}|} \sum_{x \in \mathcal{T}_{ic}} [\phi(x, 0, \theta) - h(x, 0)] \nabla_{\theta} (\phi(x, 0, \theta)) \quad (39)$$

aparecen los términos $2(\nabla_{\theta} \phi_{\theta})\phi_{\theta}$ y $2(\nabla_{\theta} \phi_{\theta})h$; ambos presentan no linealidad. Este mismo comportamiento se puede encontrar en las derivadas de gradientes de la condición de contorno. El gradiente con respecto a los parámetros del término asociado a la física,

$$\nabla_{\theta} \mathcal{L}_f(\theta^*) = \frac{1}{|\mathcal{T}_f|} \sum_{x \in \mathcal{T}_f} \nabla_{\theta} \|(\partial_{tt} - \nabla^2 + m^2)\phi(x, t, \theta)\|^2, \quad (40)$$

$$= \frac{1}{|\mathcal{T}_f|} \sum_{x \in \mathcal{T}_f} 2 \left((\partial_{tt} - \nabla^2 + m^2)\phi(x, t, \theta) \right) \left((\partial_{tt} - \nabla^2 + m^2)\nabla_{\theta} \phi(x, t, \theta) \right) \quad (41)$$

$$= \frac{2}{|\mathcal{T}_f|} K G^2 \sum_{x \in \mathcal{T}_f} \phi(x, t, \theta) \nabla_{\theta} \phi(x, t, \theta) \quad (42)$$

Las condiciones KKT expandidas,

$$\frac{2}{|\mathcal{T}_{ic}|} \sum_{x \in \mathcal{T}_{ic}} [\phi(x, 0, \theta^*) - h(x, 0)] \nabla_{\theta} \phi(x, 0, \theta^*) + \frac{2}{|\mathcal{T}_{ic}|} \sum_{x \in \mathcal{T}_{bc}} [\phi(L, t, \theta^*) - g(L, t)] \nabla_{\theta} \phi(L, t, \theta^*) \quad (43)$$

$$+ \frac{2v}{|\mathcal{T}_f|} K G^2 \sum_{x \in \mathcal{T}_f} \phi(x, t, \theta^*) \nabla_{\theta} \phi(x, t, \theta^*) = 0, \quad (44)$$

$$\mathcal{L}_f(\theta^*) = 0, \quad (45)$$

nos muestra que el optimizador si o sí debe cumplir la función de pérdida con los parámetros óptimos para este resultado. Esto implica que el optimizador debe resolver un sistema de ecuaciones no lineales de alta dimensionalidad (dimensionalidad inducida por la red neuronal),

$$\frac{2}{N_{ic}} \mathbf{J}_{ic}^T \mathbf{R}_{ic}(\theta) + \frac{2}{N_{bc}} \mathbf{J}_{bc}^T \mathbf{R}_{bc}(\theta) + \frac{2v}{N_f} K G^2 \mathbf{J}_f^T \Phi_f(\theta) = \mathbf{0}, \quad (46)$$

$$\mathcal{L}_f(\theta) = 0, \quad (47)$$

donde:

$$1. \quad N_{ic} = |\mathcal{T}_{ic}|, \quad N_{bc} = |\mathcal{T}_{bc}|, \quad N_f = |\mathcal{T}_f|.$$

2. $\mathbf{R}_{ic}(\boldsymbol{\theta}) \in \mathbb{R}^{N_{ic}}$ con entradas $\phi(\mathbf{x}_i, 0, \boldsymbol{\theta}) - h(\mathbf{x}_i, 0)$.
3. $\mathbf{J}_{ic}(\boldsymbol{\theta}) \in \mathbb{R}^{N_{ic} \times p}$ con filas $\nabla_{\boldsymbol{\theta}} \phi(\mathbf{x}_i, 0, \boldsymbol{\theta})^\top$ (análogo para $\mathbf{R}_{bc}$, $\mathbf{J}_{bc}$).
4. $\Phi_f(\boldsymbol{\theta}) \in \mathbb{R}^{N_f}$ con entradas $\phi(\mathbf{x}_k, t_k, \boldsymbol{\theta})$.
5. $\mathbf{J}_f(\boldsymbol{\theta}) \in \mathbb{R}^{N_f \times p}$ con filas $\nabla_{\boldsymbol{\theta}} \phi(\mathbf{x}_k, t_k, \boldsymbol{\theta})^\top$.
6. v es el multiplicador de Lagrange asociado a la restricción $\mathcal{L}_f(\boldsymbol{\theta}) = 0$.
7. KG es el operador diferencial de Klein-Gordon.

Esto sugiere que debe haber un multiplicador de lagrange por cada restricción. Se tendrá la misma cantidad de multiplicadores por N_f puntos en dónde debe cumplir la igualdad $\mathcal{L}_f(\boldsymbol{\theta}) = 0$; esto dependerá de si la red es lo suficientemente expresiva como para llegar a ese óptimo de pareto. Entonces, supone que la arquitectura que se eligió tenga las suficientes características como para darle existencia a ese punto óptimo. Esto es inviable para optimizadores de segundo orden donde deberá calcularse la segunda derivada con respecto a los parámetros. Cabe recalcar que la función de costo no es convexa. Toca usar métodos de optimización no convexa para este caso.

3.3.2. Minimización sin restricciones

Krishnapriyan et al. (2021), propone tratar la restricción de igualdad dura como una restricción blanda, es decir, usar una constante de regularización en la función objetivo, esto convierte el problema de optimización con restricción en un problema de optimización sin restricción Multi-Objetivo (MO). Boyd & Vandenberghe (2004), en su libro de optimización convexa, definen el problema de optimización vectorial como:

$$\begin{aligned} \min \quad & f_0(x) \\ \text{s.a} \quad & f_i(x) \leq 0, \\ & h_j(x) = 0. \end{aligned} \tag{48}$$

En dónde $f_0(x)$ y $f_i(x)$ son convexas y $h_j(x)$ es afín. También, $x \in \mathbb{R}^n$ es la variable de optimización, $K \subseteq \mathbb{R}^q$ es un cono propio, $f_0 : \mathbb{R}^n \rightarrow \mathbb{R}$ es la función objetivo, $f_i : \mathbb{R}^n \rightarrow \mathbb{R}$ son las restricciones de desigualdad, y $h_i : \mathbb{R}^p \rightarrow \mathbb{R}$ son las restricciones de igualdad. También define el conjunto de todos los valores alcanzables $\mathcal{O}$ como:

$$\mathcal{O} = \{f_0(x) \mid \exists x \in \mathcal{D}, f_i(x) \leq 0, i = 1, 2, \dots, m; h_j(x) = 0, j = 1, \dots, p\}. \tag{49}$$

Si x^* es un punto optimo, entonces $f_0(x)$, el objetivo puede compararse con cualquier otro punto factible, y debe ser mejor o igual a él. Entonces, un punto x^* es óptimo si y solo si es factible y

$$\mathcal{O} \subseteq f_0(x^*) + K. \tag{50}$$

En dónde el conjunto $f_0(x^*) + K$ es el cono propio definido centrado en $f_0(x^*)$ que define la relación de orden, es decir, quien es mayor a la función optima; el cono se define como: $K = \mathbb{R}_+^q$. Ahora, el cono me dice la región en dónde están los peores resultados. Si K me apunta hacia dónde están los peores resultados, entonces $-K$ me apunta hacia dónde están los mejores resultados, es decir, que si estoy en $f_0(x)$ y miro en la dirección $-K$ y veo que no hay ningún otro valor $f(y) \in \mathcal{O}$, entonces quiere decir que estoy en un valor de Pareto. Un valor de Pareto es un punto factible x^* y es un minimal del conjunto de valores alcanzables $\mathcal{O}$, los puntos de Pareto son aquellos que pertenecen al conjunto de Pareto definido:

$$\mathcal{P} \subseteq \mathcal{O} \cap \text{bd}\mathcal{O}. \tag{51}$$

Es decir, pertenece al borde del conjunto de valores objetivo alcanzables. De este mismo modo, introducimos el problema de minimización de Krishnapriyan et al. (2021):

$$\min_{(\boldsymbol{\theta}, \lambda)} \left(\mathcal{L}_{ic}(\boldsymbol{\theta}) + \mathcal{L}_{bc}(\boldsymbol{\theta}) \right) + \lambda_f \mathcal{L}_f(\boldsymbol{\theta}) \tag{52}$$

Si $\lambda_f = 1$, esto es lo que se conoce como “Vanilla PINN”. En dónde, $\lambda \geq_{K^*} 0$ representa todos los vectores del hiperplano que sostienen al conjunto $\mathcal{O}$ en el optimo encontrado como punto de contacto. Esto presupone un problema debido si el conjunto $\mathcal{O}$ no es convexo entonces hay posibles valores de Paretos en las regiones inalcanzables del conjunto, por lo que, la regularización no podrá encontrar todos los puntos de Pareto pero si algunos que también me minimizan la función objetivo.

La formulación del problema de optimización se puede generalizar aún más de la siguiente manera (Rohrhofer et al., 2023):

$$\min_{(\boldsymbol{\theta}, \lambda)} \sum_{i \in \{f, ic, bc\}} \lambda_i \mathcal{L}_i(\boldsymbol{\theta}) \quad (53)$$

En esta formulación, si $\lambda_f = 1$, entonces es igual a la función de pérdida propuesta por Wang et al. (2021) que regulariza las funciones de pérdida de los datos, y la función de pérdida de la física. Esto, visto de otra manera, es una reformulación a la función de pérdida de la PINN. Para tratar esto, propone la siguiente representación:

$$\min_{(\boldsymbol{\theta}, \lambda)} \mathcal{L}_f(\boldsymbol{\theta}) + \sum_{i \in \{ic, bc\}} \lambda_i \mathcal{L}_i(\boldsymbol{\theta}) \quad (54)$$

Así logra el equilibrio entre los gradientes de las funciones de pérdida de los datos y la física, respectivamente; el método de este artículo asegura que este equilibrio se logra ajustando los parámetros involucrados impidiendo que un término gobierne al otro. Para resolver esto, Wang et al. (2021) usa ADAM bajo el método de *Learning Rate Annelaing (LRA)* para evitar el problema tratado anteriormente y que Rohrhofer et al. (2023) introdujo como el problema del “frente de Pareto aparente”, que establece que usando métodos de gradiente (Newton, ADAM, SGD, etc.) alcanzan un conjunto de valores optimos locales en las funciones de pérdida, obteniendo el mismo problema de los gradientes que obtuvo Wang et al. (2021).

3.3.2.1. Condiciones KKT

Analizamos las condiciones KKT, en este caso, la única condición es la condición de primer orden para optimos locales:

$$\nabla_{\boldsymbol{\theta}} \left(\lambda_{ic} \mathcal{L}_{ic}(\boldsymbol{\theta}) + \lambda_{bc} \mathcal{L}_{bc}(\boldsymbol{\theta}) \right) + \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = 0 \quad (55)$$

$$\underbrace{\nabla_{\boldsymbol{\theta}} \left(\lambda_{ic} \mathcal{L}_{ic}(\boldsymbol{\theta}) + \lambda_{bc} \mathcal{L}_{bc}(\boldsymbol{\theta}) \right)}_{\text{Gradiente de los datos}} = \underbrace{-\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})}_{\text{Gradiente de la física}} \quad (56)$$

Obteniendo así, dos gradientes de diferentes funciones de pérdida que deben ser iguales en magnitud pero de dirección contraria. Aquí, si $\lambda_{ic} = \lambda_{bc} = 1$, se tiene el problema de desequilibrio de gradientes debido a que los gradientes de la función de pérdida física suele ser mayores y tener comportamientos más rígidos que los gradientes de los datos. Los λ de la regularización, se pueden calcular de la siguiente forma:

$$\lambda_i = \frac{\max_{\boldsymbol{\theta}} |\nabla_{\boldsymbol{\theta}} \mathcal{L}_f(\boldsymbol{\theta}_n)|}{|\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_n)|} \quad (57)$$

Dónde el numerador tiene el gradiente máximo de la función física y denominador tiene el promedio. Al usar el máximo en la función de pérdida se captura la rigidez máxima de la ecuación diferencial actuando como un límite superior que representa el peor de los escenarios y después promediado funcionando como un regularizador. En la mayoría de situaciones λ_i deberá ser mayor a uno, más sin embargo un análisis general de la posibles situaciones me indicarían lo siguiente:

- Si $\lambda_i = 1$, entonces en promedio el gradiente del dato i es igual al peor de los casos del gradiente físico, esto me daría el mismo problema que teníamos antes, como consecuencia la optimización no podrá salir de la región de valores de Pareto aparente (situación ideal).
- Si $\lambda_i < 1$, entonces en promedio el gradiente del dato i es mayor al peor caso del gradiente de la optimización, como consecuencia le daría más peso al este gradiente de la función de dato i , y así el optimizador exploraría otra región de Pareto con minimales inexplorados.
- Si $\lambda_i > 1$, entonces el máximo del gradiente de la física es mayor al promedio del gradiente del dato i , como consecuencia estaría en una región de Pareto más reducida.

3.3.2.2. Análisis de rigidez (Stiffness)

Como vimos antes, los gradientes de la función física son mayores y más rígidos. Esto se mencionó sin analizar a profundidad. Para su análisis, Wang et al. (2021), usó la función de pérdida de la *Vanilla PINN* y el método *steepest gradient* como optimizador:

$$\boldsymbol{\theta}_{n+1} = \boldsymbol{\theta}(n) - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}_n) \quad (58)$$

$$= \boldsymbol{\theta}(n) - \eta [\nabla_{\boldsymbol{\theta}} \mathcal{L}_f(\boldsymbol{\theta}_n) + \nabla_{\boldsymbol{\theta}} \mathcal{L}_{data}(\boldsymbol{\theta}_n)] \quad (59)$$

En dónde η es la tasa de aprendizaje o el paso. Aplican expansión de la serie de Taylor hasta el término de segundo orden,

$$\mathcal{L}(\theta_{n+1}) = \mathcal{L}(\theta_n) + (\theta_{n+1} - \theta_n)^T \cdot \nabla_{\theta} \mathcal{L}(\theta_n) + \frac{1}{2} (\theta_{n+1} - \theta_n)^T \nabla_{\theta}^2 \mathcal{L}(\xi) (\theta_{n+1} - \theta_n) \quad (19) \quad (60)$$

Si usamos la siguiente relación $\theta_{n+1} - \theta_n = -\eta \nabla_{\theta} \mathcal{L}(\theta)$ y reemplazamos en la expansión de Taylor, encontramos la diferencia que hay entre el paso $n + 1$ y n está dada por:

$$\mathcal{L}(\theta_{n+1}) - \mathcal{L}(\theta_n) = -\eta \nabla_{\theta_n} \mathcal{L}(\theta)^T \cdot \nabla_{\theta_n} \mathcal{L}(\theta_n) + \frac{1}{2} \eta^2 \nabla_{\theta} \mathcal{L}(\theta_n)^T \nabla_{\theta}^2 \mathcal{L}(\xi) \nabla_{\theta} \mathcal{L}(\theta_n) \quad (61)$$

$$= -\eta \|\nabla_{\theta_n} \mathcal{L}(\theta)\|_2^2 + \frac{1}{2} \eta^2 \nabla_{\theta} \mathcal{L}(\theta_n)^T \nabla_{\theta}^2 \mathcal{L}(\xi) \nabla_{\theta} \mathcal{L}(\theta_n) \quad (62)$$

$$= -\eta \|\nabla_{\theta_n} \mathcal{L}(\theta)\|_2^2 + \frac{1}{2} \eta^2 \nabla_{\theta} \mathcal{L}(\theta_n)^T (\nabla_{\theta}^2 \mathcal{L}_f(\xi) + \nabla_{\theta}^2 \mathcal{L}_{data}(\xi)) \nabla_{\theta} \mathcal{L}(\theta_n) \quad (63)$$

$$= -\eta \|\nabla_{\theta_n} \mathcal{L}(\theta)\|_2^2 + \frac{1}{2} \eta^2 \nabla_{\theta} \mathcal{L}(\theta_n)^T \nabla_{\theta}^2 \mathcal{L}_f(\xi) \nabla_{\theta} \mathcal{L}(\theta_n) + \frac{1}{2} \eta^2 \nabla_{\theta} \mathcal{L}(\theta_n)^T \nabla_{\theta}^2 \mathcal{L}_{data}(\xi) \nabla_{\theta} \mathcal{L}(\theta_n) \quad (64)$$

. Vale analizar cada término acá, el término $\eta \|\nabla_{\theta_n} \mathcal{L}(\theta)\|_2^2$ me representa el valor del gradiente al cuadrado por la tasa de aprendizaje, normalmente es pequeña;

$$\nabla_{\theta} \mathcal{L}(\theta_n)^T \nabla_{\theta}^2 \mathcal{L}(\xi) \nabla_{\theta} \mathcal{L}(\theta_n) = \frac{\|\nabla_{\theta} \mathcal{L}(\theta_n)\|^2}{\|\nabla_{\theta} \mathcal{L}(\theta_n)\|^2} \left(\nabla_{\theta} \mathcal{L}(\theta_n)^T \nabla_{\theta}^2 \mathcal{L}(\xi) \nabla_{\theta} \mathcal{L}(\theta_n) \right) \quad (65)$$

$$= \|\nabla_{\theta} \mathcal{L}(\theta_n)\|^2 \left(\frac{\nabla_{\theta} \mathcal{L}(\theta_n)^T}{\|\nabla_{\theta} \mathcal{L}(\theta_n)\|} \nabla_{\theta}^2 \mathcal{L}(\xi) \frac{\nabla_{\theta} \mathcal{L}(\theta_n)}{\|\nabla_{\theta} \mathcal{L}(\theta_n)\|} \right). \quad (66)$$

Por el teorema de descomposición de autovalores (Trefethen & Bau, 1997), una matriz cuadrada y de columnas independientes entre sí, puede descomponerse en una matriz Q ortogonal que contiene sus autovectores y una matriz $diag(\lambda_1, \lambda_2, \dots, \lambda_N)$, de este modo $\nabla_{\theta} \mathcal{L}(\theta) = Q^T \mathbf{diag}(\lambda_1, \lambda_2, \dots, \lambda_N) Q$, de este modo la expresión anterior se puede analizar:

$$\nabla_{\theta} \mathcal{L}(\theta_n)^T \nabla_{\theta}^2 \mathcal{L}(\xi) \nabla_{\theta} \mathcal{L}(\theta_n) = \|\nabla_{\theta} \mathcal{L}(\theta_n)\|^2 (x^T Q^T \mathbf{diag}(\lambda_1, \lambda_2, \dots, \lambda_N) Q x) \quad (67)$$

$$= \|\nabla_{\theta} \mathcal{L}(\theta_n)\|^2 (y^T \mathbf{diag}(\lambda_1, \lambda_2, \dots, \lambda_N) y) \quad (68)$$

$$= \|\nabla_{\theta} \mathcal{L}(\theta_n)\|^2 \sum_{i=1}^N \lambda_i y_i^2. \quad (69)$$

En dónde, $y = xQ$, $x = \frac{\nabla_{\theta} \mathcal{L}(\theta_n)}{\|\nabla_{\theta} \mathcal{L}(\theta_n)\|}$, Q es una matriz ortogonal que diagonaliza $\nabla_{\theta}^2 \mathcal{L}(\xi)$, y $\lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_N$ son los autovalores de $\nabla_{\theta}^2 \mathcal{L}(\xi)$. Con este mismo procedimiento y la descomposición de $\nabla_{\theta}^2 \mathcal{L}(\xi)$ en las funciones de pérdida física y de los datos se encuentran los autovalores respectivos. Partiendo del resultado de la ecuación (90) y agregandolo a la ecuación (83), obtenemos:

$$\mathcal{L}(\theta_{n+1}) - \mathcal{L}(\theta_n) = -\eta \|\nabla_{\theta} \mathcal{L}(\theta)\|^2 + \frac{1}{2} \eta \|\nabla_{\theta} \mathcal{L}(\theta)\|^2 \sum_{i=1}^N \lambda_i y_i^2 \quad (70)$$

$$= \eta \|\nabla \mathcal{L}\|^2 \left(-1 + \frac{1}{2} \eta \sum_{i=1}^N \lambda_i y_i^2 \right). \quad (71)$$

Para que el método de optimización funcione, se quiere que $\mathcal{L}_f(\theta_{n+1}) < \mathcal{L}_f(\theta_n)$, por lo tanto el lado izquierdo de la ecuación (91) debe ser negativo, para esto los términos dentro del paréntesis debe ser negativos y cumplir $1 > \frac{1}{2} \eta \sum_{i=1}^N \lambda_i y_i^2$. Esto solo es posible si los valores propios son pequeños (la función de pérdida cercana al mínimo es suave y bien comportada), sin embargo, si es complicada, tenemos que los valores propios son grandes y en vez de seguir el camino de descenso se aleja saltando a otro valle. En detalle, se descompone la función de pérdida en multiples funciones objetivos:

$$\mathcal{L}_f(\theta_{n+1}) - \mathcal{L}_f(\theta_n) = \eta \|\nabla \mathcal{L}_f(\theta)\|^2 \left(-1 + \frac{1}{2} \eta \sum_{i=1}^N \lambda_i^f y_i^2 \right), \quad (72)$$

$$\mathcal{L}_{ic}(\theta_{n+1}) - \mathcal{L}_{ic}(\theta_n) = \eta \|\nabla \mathcal{L}_{ic}(\theta)\|^2 \left(-1 + \frac{1}{2} \eta \sum_{i=1}^N \lambda_i^{ic} y_i^2 \right), \quad (73)$$

$$\mathcal{L}_{bc}(\theta_{n+1}) - \mathcal{L}_{bc}(\theta_n) = \eta \|\nabla \mathcal{L}_{bc}(\theta)\|^2 \left(-1 + \frac{1}{2} \eta \sum_{i=1}^N \lambda_i^{bc} y_i^2 \right). \quad (74)$$

Acá, de acuerdo a Wang et al. (2021) la causante del desequilibrio del gradiente está en los autovalores λ_i^f asociados a la función de pérdida física, siendo estos muy grandes cercano a un minimal.

3.3.2.3. Análisis de cotas

Wang et al. (2021), desarrolla el análisis de cotas a partir de las desigualdades de Sobolev a $\nabla_{\theta}\mathcal{L}_f$ y $\nabla_{\theta}\mathcal{L}_i$:

$$\|\nabla_{\theta}\mathcal{L}_f(\theta)\|_{L^{\infty}} \leq C\|\nabla_{\theta}^2\mathcal{L}_f(\theta)\|_{L^{\infty}}, \quad (75)$$

$$\|\nabla_{\theta}\mathcal{L}_i(\theta)\|_{L^{\infty}} \leq C\|\nabla_{\theta}^2\mathcal{L}_i(\theta)\|_{L^{\infty}}. \quad (76)$$

Esto nos impone que el máximo valor del gradiente está acotado por el máximo valor de su curvatura de la función de pérdida, conectado con los valores propios del Hessiano de la función de pérdida, usaremos la relación (2.3.11) del libro Numerical linear algebra de Trefethen & Bau (1997), sea el Hessiano $H \in \mathbb{R}^{N \times N}$ dónde N es el número de *puntos de colocación* y $(1/\sqrt{N})\|H\|_{\infty} \leq \|H\|_2 \leq \sqrt{N}\|H\|_{\infty}$, entonces las desigualdades de Sobolev tendrían relación directa con los autovalores del Hessiano de las funciones de pérdida físicas y de los datos;

$$\|\nabla_{\theta}\mathcal{L}_f(\theta)\|_{L^{\infty}} \leq C\sqrt{N}\|\nabla_{\theta}^2\mathcal{L}_f(\theta)\|_{L^2} = C\sqrt{N}\sqrt{\text{máx } \lambda_f(H_f^T H_f)}, \quad (77)$$

$$\|\nabla_{\theta}\mathcal{L}_{data}(\theta)\|_{L^{\infty}} \leq C\sqrt{N}\|\nabla_{\theta}^2\mathcal{L}_{data}(\theta)\|_{L^2} = C\sqrt{N}\sqrt{\text{máx } \lambda_{data}(H_{data}^T H_{data})}. \quad (78)$$

Así que el valor del gradiente está acotado por el máximo autovalor de su curvatura. Esto implica que un autovalor grande de $H^T H$ habilita la existencia de valores grandes del gradiente, mientras que un autovalor pequeño restringe el gradiente a valores pequeños. Por lo tanto, se puede reafirmar el desequilibrio entre gradientes.

3.4. Learning rate annealing for PINN

Partimos de la función objetivo de la ecuación 76, y minimizamos con respecto a un método de primer orden en el gradiente:

$$\theta_{n+1} = \theta_n - \eta \left[\nabla_{\theta}\mathcal{L}_f(\theta) + \sum_{i \in \{ic, bc\}} \lambda_i \nabla_{\theta}\mathcal{L}_i(\theta) \right] \quad (79)$$

Se calcula los pesos de las funciones de pérdida de las condiciones iniciales y de contorno usando la ecuación (78), y se actualiza usando una función de media móvil para eliminar el ruido generado por los gradientes en el entrenamiento:

$$\lambda_i = (1 - \alpha)\lambda_i + \alpha\hat{\lambda}_i. \quad (80)$$

Para posteriormente actualizar los pesos bajo la ecuación (100) con los nuevos lambdas. El algoritmo propuesto por Wang et al. (2021), está dado por el algoritmo 2 con la modificación de no usar SGD sino Adam como optimizador como una aproximación al Gradiente Natural de Fisher.

Algorithm 1 Algoritmo de LRA + ADAM

Require: Hiperparámetros recomendados: $\eta = 10^{-3}$, $\alpha = 0.9$.

Require: Número de pasos S , número de restricciones M .

Require: Pesos iniciales λ_i para cada término de pérdida $i = 1, \dots, M$.

Bucle de Entrenamiento

1: **for** $n \leftarrow 1$ **to** S **do**

(a) Cálculo de estadísticas de gradiente

2: Calcular el máximo gradiente del residuo físico:

3: $G_{max} \leftarrow \text{máx}_{\theta} \{|\nabla_{\theta}\mathcal{L}_r(\theta_n)|\}$

4: Calcular la media del gradiente para cada término i :

5: $G_{mean}^i \leftarrow |\nabla_{\theta}\mathcal{L}_i(\theta_n)|$

6: $\hat{\lambda}_i \leftarrow \frac{G_{max}}{G_{mean}^i}$ ▷ (Ec. 78)

(b) Actualización de pesos (Promedio Móvil)

7: Actualizar λ_i usando el factor de suavizado α :

8: $\lambda_i \leftarrow (1 - \alpha)\lambda_i + \alpha\hat{\lambda}_i$ ▷ (Ec. 101)

(c) Actualización de Parámetros de la Red

9: Calcular gradiente total ponderado:

10: $\nabla_{total} \leftarrow \nabla_{\theta}\mathcal{L}_r(\theta_n) + \sum_{i=1}^M \lambda_i \nabla_{\theta}\mathcal{L}_i(\theta_n)$

11: Aplicar ADAM:

12: $\theta_{n+1} \leftarrow \theta_n - \eta \nabla_{total}$ ▷ (Ec. 100)

13: **end for**

14: **return** Parámetros optimizados θ_{S+1}

3.5. Causal Training

Hasta ahora, hemos atacado el problema de rigidez de gradientes que induce al optimizador a caer en mínimos locales, sin embargo no hemos atacado el problema de las EDPs hiperbólicas como el sesgo de espectro y propagación de los frentes de onda. El Causal training introduce una venda a la función de pérdida asociada a la física obligandola a centrarse en el presente para seguir al futuro (Wang et al., 2022). Debido a que los fenómenos de propagación dependen del pasado, la red neuronal debe respetar este proceso.

$$\mathcal{L}_f = \frac{1}{N_t} \sum_{i=0}^{N_t} w_i \mathcal{L}_f(\theta, t_i). \quad (81)$$

En donde,

$$w_i = \exp \left(-\epsilon \sum_{k=1}^{i-1} \mathcal{L}_f(\theta, t_k) \right) \quad (82)$$

Esto impone un factor de penalización ϵ , dándole prioridad al presente y ocultando parte del pasado. Wang et al. (2022) proponen valores de epsilon $\{10^{-2}, 10^{-1}, 10^0, 10^1, 10^2\}$.

3.6. Muestreo Estocástico

Debido a que el dominio $D \subset \mathcal{R}^n$ es grande y el tiempo de computo se complejiza a la medida que haya más datos, entonces se utiliza un esquema de muestreo estocastico cada época de N_f, N_{bc}, N_{ic} puntos. Debido a la rigidez de los gradientes, se asigna más puntos de colocación a la física que a las condiciones iniciales y de frontera. Se usa un esquema de muestreo localizado, debido a que las EDPs de dispersión presentan frentes de onda, entonces la física sucederá dentro de estos frentes de onda, y el exterior deberá cumplir las condiciones iniciales. Por tal motivo, se usa 80 % de los puntos de colocación de la física dentro del frente de onda y 20 % afuera de la onda para garantizar que la red aún así no viole la física con la condición inicial. Para esto, se usó la restricción de cono de luz de Einstein debido a que a la ecuación de Klein-Gordon.

3.7. ADAM

Para entender el método Adam, se debe partir del *Gradiente Natural de Fisher* (Pascanu & Bengio, 2013;), esto quiere decir que el gradiente ya no se movera por el espacio euclideo común y corriente sino que se movera através del espacio de distribución de probabilidades bajo la métrica de Kullback-Leibler (KL) que tiene para cada camino. Para esto, se debe introducir la matriz de fisher en el método de gradiente descendente:

$$\theta_{n+1} = \theta_n - \eta \underbrace{F^{-1}}_{\text{Matriz de fisher}} \nabla_{\theta} \mathcal{L}(\theta) \quad (83)$$

La matriz de Fisher contiene las varianzas y covarianzas de los diferentes pasos estocásticos generados por los *minibatches* del método del descenso del gradiente. Los autores Pascanu & Bengio (2013) definieron la matriz de Fisher:

$$F = \mathbb{E} \left[\underbrace{\nabla_{\theta} \text{Log}(P(x|\theta)) \cdot \nabla_{\theta} \text{Log}(P(x|\theta))^T}_{\text{Segundo momento}} \right] \quad (84)$$

De acuerdo con la ecuación (102), la matriz de Fisher es el segundo momento de $\nabla_{\theta} \text{Log}(P(x|\theta))$, lo que parece ser el gradiente de una típica función de pérdida para una función de probabilidad (Amari, 1998). El método de Adam, asume que los parámetros son independientes entre sí y por tal motivo solo sobreviven los elementos de la diagonal de Fisher, y así el método del descenso es una aproximación al método del Gradiente Natural de Fisher ():

$$\theta_{n+1} = \theta_n - \eta \underbrace{\text{diag}(F^{-1})}_{\text{Varianza}} \underbrace{\nabla_{\theta} \mathcal{L}(\theta)}_{\text{Primer momento}}. \quad (85)$$

Aparte de esto, Adam conserva el decaimiento exponencial del momento como lo hace el método de Momento de una manera similar (Ruder, 2016), y anexa el segundo momento como cercanía al método Natural de Fisher:

$$\theta_{n+1} = \theta_n - \eta \frac{m_t}{\sqrt{v_t}} \quad (86)$$

En dónde:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} \mathcal{L}(\theta), \quad (87)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} \mathcal{L}(\theta))^2. \quad (88)$$

Acá Adam (2014) aclaró que usó la raíz cuadrada del inverso de la matriz de varianza debido a que conservaría invarianza en la escala del gradiente y es un preconditionador más conservador, es decir, en el método de Gradiente Natural de Fisher, si la curvatura de Fisher es pequeña entonces asume que está en una maceta y da un paso muy grande para poder atravesarla, debido a que tenemos funciones no convexas esto puede ser omitir minimos locales, así que para mitigar este problema no usa la varianza total sino la raíz cuadrada de la varianza, algo así como un desviación estandard.

4. Arquitectura

4.1. Normalización de las variables

La normalización de las variables permite comparar a la misma escala las variables, asegurando una contribución uniforme en el entrenamiento de aprendizaje de la NN, por tal motivo conduce a resultados de aprendizaje más precisos (Xu et al., 2025). El proceso de Normalización que se usó fue el método de "min-max" modificado para escalarlo al rango de $[-1, 1]$:

$$x_{norm} = 2 \frac{x - x_{min}}{x_{max} - x_{min}} - 1, \quad (89)$$

$$t_{norm} = 2 \frac{t - t_{min}}{t_{max} - t_{min}} - 1. \quad (90)$$

Al usar este proceso de escalador, podemos ver que hay una constante de escalamiento:

$$x_{norm} = \alpha_x (x - x_{min}) - 1, \quad (91)$$

$$t_{norm} = \alpha_t (t - t_{min}) - 1. \quad (92)$$

En dónde $\alpha_x = 2/(x_{max} - x_{min})$ Y $\alpha_t = 2/(t_{max} - t_{min})$. Ahora, como las funciones de pérdida expresan la física o el fenómeno en los dominios originales, entonces existe un escalamiento α_i que:

$$\frac{\partial}{\partial x} = \alpha_x \frac{\partial}{\partial x_{norm}}, \quad \frac{\partial^2}{\partial x^2} = \alpha_x^2 \frac{\partial^2}{\partial x_{norm}^2}, \quad (93)$$

$$\frac{\partial}{\partial t} = \alpha_t \frac{\partial}{\partial t_{norm}}, \quad \frac{\partial^2}{\partial t^2} = \alpha_t^2 \frac{\partial^2}{\partial t_{norm}^2}. \quad (94)$$

Y así obtener, la función de pérdida de la física se debe escalar de esta manera, al igual que la función de pérdida de la condición inicial y la condición de contorno para mantener el dominio de la física original en las escalas de $x, t = [-1, 1]$.

$$(\alpha_t^2 \partial_{tt} - \alpha_x^2 \nabla^2 + m^2) \phi(\mathbf{x}_{norm}, \mathbf{t}_{norm}; \theta) = 0 \quad (95)$$

4.2. Función de activación - SIREN

Para el entrenamiento de la PINN de EDP periódicas o con solución aproximada de la serie de Fourier, Sitzmann et al. (2020) propone la metodología SIREN con funciones de activación sinusoidales:

$$\Phi(\mathbf{x}) = \mathbf{W}_n (\phi_{n-1} \circ \phi_{n-2} \circ \dots \circ \phi_0)(\mathbf{x}) + \mathbf{b}_n, \quad \phi_i(\mathbf{x}_i) = \sin(\mathbf{W}_i \mathbf{x}_i + \mathbf{b}_i). \quad (96)$$

El autor argumentó que al usar SIREN debe realizarse ajustes en la metodología de iniciación de los parámetros, esto es debido a la linealidad que tiene el seno en valores cercanos a su cero, si los pesos se inician en valores muy pequeños, entonces la función de activación perdería su poder de representación para funciones continuas. Para esto, analizaron la distribución de señal de salida en cada capa. En la capa input se asume que la distribución es uniforme, al ser procesada por la función de activación sinusoidal la señal de salida cambia a una distribución arcoseno; al tener varias capas la distribución U (arcoseno) por el teorema del límite central se va volviendo una distribución normal. Los autores proponen que para tener estabilidad se debe controlar la varianza de la distribución U en las capas finales de las capas ocultas. Para la elección de los pesos, propusieron:

$$w_i \sim \mathcal{U}\left(-\sqrt{\frac{6}{n}}, \sqrt{\frac{6}{n}}\right), \quad (97)$$

donde n es el número de entradas de la neurona. Ahora para la atacar el problema de valores pequeños (bajas frecuencias), se propone un factor w_0 grande que escale los frecuencias y así pueda tener la representación de la función seno, transformando la función sinusoidal a

$$\sin(w_0 \mathbf{W}_i \mathbf{x}_i + \mathbf{b}_i). \quad (98)$$

El paper recomienda un valor $w_0 = 20$ para EDPs con necesidad de expresividad.

4.3. Función de activación - Racional

La necesidad de encontrar una función de activación que sea expresiva, que sea diferenciable de orden superior C^n con $n \geq 2$, y cuyas derivadas no se apaguen rápidamente, nos llevó a considerar las funciones de activación racionales. En particular, la forma $f(x) = \frac{1}{1+x^2}$ (una función racional de tipo campana) ha sido estudiada como una aproximación eficiente a la sigmoide y como caso particular de una familia más amplia de funciones racionales suaves (Boulle N., 2020).

Trabajos como el de Boullé et al. (2020) demostraron que las redes neuronales con funciones de activación racional (RAF, por sus siglas en inglés) pueden aproximar funciones con una profundidad exponencialmente menor que las redes ReLU tradicionales, manteniendo una suavidad C^∞ (es decir, derivables infinitas veces) y evitando que sus derivadas se anulen rápidamente, lo que mitiga el problema del gradiente evanescente. Posteriormente, variantes mejoradas como ERA abordaron la inestabilidad numérica de las RAF originales, haciéndolas prácticas en tareas de clasificación de imágenes y reconstrucción 3D (Dubey et al., 2022).

Por estas propiedades: expresividad, diferenciabilidad de orden superior y derivadas no evanescentes, la función racional $\frac{1}{1+x^2}$ y sus generalizaciones resultan candidatas ideales para contextos donde se requieren entrenamientos estables y aproximaciones eficientes.

4.4. Embedding Fourier Transform

Para mejorar la capacidad de las redes neuronales en la aproximación de funciones de alta frecuencia o multi-escala, se han propuesto transformaciones de las coordenadas de entrada basadas en características de Fourier. La técnica más común es el random Fourier features (Tancik et al., 2020), que consiste en aplicar un mapeo

$$\gamma(\mathbf{v}) = \begin{bmatrix} \cos(\mathbf{B}\mathbf{v}) \\ \sin(\mathbf{B}\mathbf{v}) \end{bmatrix},$$

donde cada entrada de $\mathbf{B} \in \mathbb{R}^{m \times d}$ se muestrea idenpendientes e idénticamente distribuidas de una distribución gaussiana $\mathcal{N}(0, \sigma^2)$. Este mapeo permite que redes totalmente conectadas aprendan altas frecuencias de manera más efectiva, mitigando el sesgo espectral.

En el contexto de las PINNs, wang et al. (2021) extiende esta idea proponiendo embeddings multi-escala de Fourier para darle representación fenómenos de onda aprovechando su separabilidad de variables. Se utilizan múltiples mapeos con diferentes valores de (σ_i) que se aplican por separado a las coordenadas espaciales y temporales. Luego, las salidas de cada rama entran a una MPL poco profundo pero ancho (200 neuronas por capa) para obtener así la representación de las bases de Fourier ($\{\sin, \cos\}$); por último, la salida de la red se combinan mediante multiplicación punto a punto para luego ser procesada por la capa de outlayer. Esta estrategia permite capturar comportamientos periódicos o rápidamente variables en diferentes escalas, como se demuestra en ecuaciones de onda, reacción-difusión y convección.

Tanto PirateNets como en Multi-Sacale Fourier incorporan estos embeddings como primer paso de su flujo hacia adelante, utilizando típicamente características aleatorias de Fourier con un único σ (ajustable) o múltiples escalas.

4.5. Arquitecturas

A continuación se describen cinco arquitecturas representativas utilizadas en la literatura de PINNs.

4.5.1. PINN Vanilla (original)

La formulación original de las Physics-Informed Neural Networks (Raissi, Perdikaris y Karniadakis, 2019) aproxima la solución $u(t, \mathbf{x})$ mediante un perceptrón multicapa (MLP) totalmente conectado con funciones de activación tanh. La red se entrena minimizando una pérdida a partir de soft-constraint con pesos de $\lambda = [1.0, 1.0, 1.0]$:

$$\mathcal{L}(\theta) = \mathcal{L}_{ic}(\theta) + \mathcal{L}_{bc}(\theta) + \mathcal{L}_r(\theta),$$

donde $\mathcal{L}_{ic}$ y $\mathcal{L}_{bc}$ castigan el incumplimiento de las condiciones iniciales y de contorno, y $\mathcal{L}_r$ impone el residuo de la EDP. Esta arquitectura sufre de rigidez de gradiente, progradación de la red y sesgo espectral. Funciona bien para problemas suaves y de baja frecuencia, pero falla en soluciones no lineales, funciones de altas y bajas frecuencias, y en EDPs hiperbólicas por su incapacidad de captar la propagación de la red como a su vez de desvanecimiento del gradiente en alguno de los términos involucrados.

4.5.2. Scaling Fourier Feature (multi-escala)

Para abordar el sesgo espectral, Wang, Wang y Perdikaris (2021) proponen dos arquitecturas basadas en características de Fourier. La primera (*multi-scale Fourier feature architecture*) aplica múltiples mapeos $\gamma^{(i)}$ con diferentes σ_i a la entrada, pasa cada uno por la misma red totalmente conectada y concatena las salidas antes de una capa lineal final. La segunda (*spatio-temporal multi-scale Fourier feature architecture*) trata por separado las coordenadas espaciales y temporales, aplica distintos mapeos de Fourier a cada grupo, los procesa en paralelo y luego los combina mediante multiplicación punto a punto. Estas arquitecturas no añaden parámetros entrenables adicionales y permiten aprender componentes de alta frecuencia que la PINN vanilla no puede capturar. La arquitectura de la red es una Vanilla PINN.

4.5.3. Modified MLP (M4) de Wang

En Wang, Teng y Perdikaris (2021) se identifican las patologías de gradientes desbalanceados en PINNs y se proponen dos mejoras: (i) un algoritmo de *annealing* de la tasa de aprendizaje que ajusta automáticamente los pesos de los términos de la pérdida (λ_i en $\mathcal{L} = \mathcal{L}_r + \sum \lambda_i \mathcal{L}_i$) basándose en la magnitud de los gradientes; (ii) una arquitectura mejorada (denominada M4) que incorpora unidades adaptativas con compuertas y conexiones residuales. La propagación directa de M4 es:

$$\begin{aligned} U &= \phi(XW^1 + b^1), \quad V = \phi(XW^2 + b^2), \\ H^{(1)} &= \phi(XW^{z,1} + b^{z,1}), \\ Z^{(k)} &= \phi(H^{(k)}W^{z,k} + b^{z,k}), \\ H^{(k+1)} &= (1 - Z^{(k)}) \odot U + Z^{(k)} \odot V, \\ f_\theta(x) &= H^{(L+1)}W + b. \end{aligned}$$

Esta arquitectura introduce multiplicaciones elemento a elemento entre las representaciones de las compuertas y las activaciones, lo que reduce la rigidez del flujo de gradiente y mejora la precisión en problemas como la ecuación de Helmholtz y Klein–Gordon.

4.5.4. PirateNet (Physics-Informed Residual Adaptive Network)

Propuesta por Wang, Li, Chen y Perdikaris (2024), PirateNet está diseñada para permitir el entrenamiento estable de redes profundas en PINNs. La arquitectura combina:

- *Embedding* de Fourier aleatorio inicial.
- Compuertas U y V obtenidas de dos capas densas a partir del *embedding*.
- Skip connection a través de bloques residuales adaptativos: cada bloque contiene tres capas densas con activaciones, dos operaciones de compuerta (usando U y V), y una conexión residual con un parámetro entrenable $\alpha^{(l)}$:

$$\mathbf{x}^{(l+1)} = \alpha^{(l)} \mathbf{h}^{(l)} + (1 - \alpha^{(l)}) \mathbf{x}^{(l)}.$$

La clave está en inicializar $\alpha^{(l)} = 0$, de modo que al inicio la red se comporta como una combinación lineal del *embedding* (una red superficial). Durante el entrenamiento, los $\alpha^{(l)}$ crecen para introducir no linealidad de manera progresiva. Además, la última capa se inicializa resolviendo un problema de mínimos cuadrados que ajusta los datos disponibles (condiciones iniciales, solución de la EDP linealizada, etc.). Esta estrategia evita las patologías de inicialización de las derivadas profundas y permite que PirateNet escale bien con la profundidad, superando a MLP y ResNet convencionales en benchmarks como Allen–Cahn, KdV, Grey–Scott y cavidad *lid-driven*.

4.5.5. AbuPINN (Adaptive Blending Units for PINNs)

Basada en Wang, Lu, Song y Huang (2023), AbuPINN aprende la función de activación óptima para cada problema mediante una combinación lineal de funciones candidatas. La formulación es:

$$f(x) = \sum_{i=1}^N G(\alpha_i) \sigma_i(\beta_i x),$$

donde σ_i son funciones elementales (sin, tanh, GELU, Swish, Softplus, etc.), α_i son coeficientes aprendibles normalizados mediante softmax, y β_i son factores de escala adaptativos. Esta unidad de activación se puede aplicar de manera neuronal o por capa. AbuPINN garantiza derivadas continuas y evita inestabilidades numéricas. A su vez, intenta ablandar la complejidad del paisaje de pérdida dándole expresividad a la red a partir de las características de cada función involucrada en la función de activación f . En los experimentos, AbuPINN supera consistentemente a las funciones fijas (tanh, sin, etc.) y a otras activaciones adaptativas en problemas de convección, Burgers, Allen–Cahn, KdV, Cahn–Hilliard y Navier–Stokes. Además, se muestra que el kernel tangente neural resultante es aprendible y presenta un espectro de autovalores más favorable para la convergencia.

4.5.6. Modified Adaptive Blending Units Residual for PINNs

Basada en la expresividad de la AbuPINN y la capacidad de representabilidad de las PirateNet a problemas no lineales, se construye una *Modified Adaptive Blending Units Residual for PINNs*. Tanto como la PirateNet y la MultiScaling Layers requieren de Fourier Transform Embedding, esto sesga la red a un hiperparámetro σ , lo cuál parece ser que AbuPINN con SIREN para cos y sin permite que la red aprenda los sesgos necesarios para su expresividad. Esto nos da el interés de unir ambas características: el aprendizaje de los sesgos de la red y la expresividad de la red a través de conexiones residuales y convexas. Utilizamos un conjunto de funciones de activación {sin, cos, tanh, racional} con el fin de representar las bases de Fourier, las condiciones iniciales y cambios fuertes en los frentes de onda. Se hicieron las conexiones residuales y convexas (U, V) cada 3 capas.

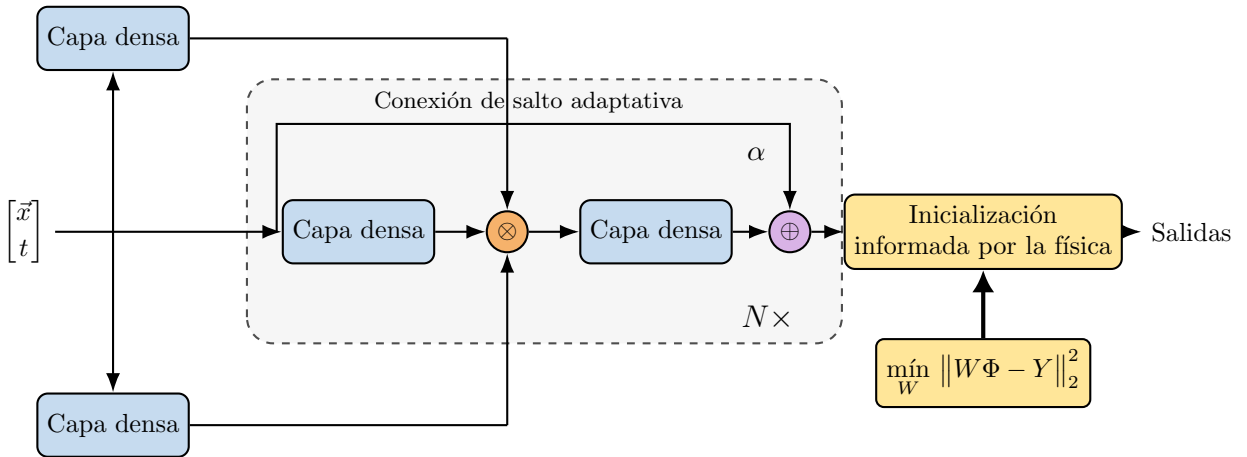


Figura 2: Arquitectura de la red empleada, basada en PirateNet Wang et al. (2024), sin la etapa de *embedding* de coordenadas: la entrada $(\vec{x}, t)$ alimenta directamente las capas de compuerta U y V y el tronco residual. Cada capa densa consta de 3 capas con 64 neuronas cada una, cada neurón contiene un Abu con {sin, cos, tanh, racional}.

5. Experimentación

5.1. Metodología

Se replican cada una de estas redes neuronales, obteniendo errores de optimización con respecto a las redes propuestas en este artículo. Con esto, se implementan en la ecuación de Klein-Gordon para analizar su comportamiento. Todas las redes neuronales bajo la implementación de la EDP de interés se entrenaron con algoritmos de Escalamiento de inputs, LRA, Causal Training, Muestreo estocástico focalizado, decaimiento exponencial de la tasa de aprendizaje y precisión mixta: 32-Float solo para parámetros de la red y 64-Float para puntos de colocación, coeficientes de optimización, gradientes de retropropagación, etc.

Los hiperparámetros usados en todas las redes son:

- Tasa de aprendizaje inicial: 1×10^{-3}
- Decaimiento de la tasa: 0.9 cada 2 000 épocas
- Número de épocas (iteraciones): 100 000
- Puntos de colocación de la física (N_f): 12 000
- Puntos de condición inicial (N_{ic}): $N_f/2 = 6 000$

- Puntos de condición de contorno (N_{bc}): $N_f/4 = 3\,000$
- α del *learning rate annealing* (LRA): 0.9
- Coeficientes del LRA iniciales: [1.0, 1.0, 0.1]
- $\omega_0 = 15.0$ para las arquitecturas que usaron función de activación sin o cos (SIREN, AbuPINN, MABUR-PINN).
- Optimizador: Adam

Cuadro 1: Arquitecturas.

Arquitectura	Profundidad	Ancho	Épocas	ϵ	Chunks	Otros
PINN Vanilla (clásica)	6	64	100k	10	16	—
PINN (SIREN)	6	64	100k	10	16	Función de activación SIREN
Scaling Fourier Feature	5	200	100k	10	16	Embedding múltiple σ
Modified MLP (M4)	6	64	100K	10	16	Arq. con compuertas + Embedding
AbuPINN	6	64	100k	100	16	Activación combinada
MABUR-PINN	6 (2 bloques)	64	100K	100	16	M4 + Abu + residual adaptativo

6. Resultados y Discusiones

Para evaluar el desempeño de las distintas arquitecturas descritas, se replicaron y se entrenaron modelos bajo las mismas condiciones experimentales, utilizando los hiperparámetros comunes reportados en la sección anterior. Las métricas consideradas fueron: el error relativo L_2 , el error absoluto medio, el número total de parámetros entrenables y el valor final de la función de costo (pérdida compuesta) al término del entrenamiento. Los resultados se resumen en la Tabla 2. Se observa que ABU-PINN alcanza el error más bajo (0.02 en L_2 relativo) con una cantidad moderada de parámetros (42 073) y el costo final más pequeño (4.60×10^{-7}). En contraste, la PINN clásica presenta el peor desempeño (0.71), mientras que MSF-PINN, a pesar de su alto número de parámetros (2.11×10^6), no logra reducir significativamente el error. Estos resultados sugieren que las estrategias de aprendizaje de funciones de activación adaptativas (ABU-PINN) y la combinación con conexiones residuales (MABUR-PINN) ofrecen ventajas claras en precisión y eficiencia computacional. Sin embargo, la MABUR-PINN no logra superar a las redes neuronales M4-PINN y ABU-PINN.

Cuadro 2: Resultados comparativos de las arquitecturas.

Arquitectura	Error rel. L_2	Error abs.	Num. de parámetros	Costo final
ABU-PINN	0.02	0.0008	42 073	4.60×10^{-7}
M4-PINN	0.04	0.0013	124 033	2.21×10^{-6}
MABUR-PINN	0.11	0.0037	51 177	7.25×10^{-6}
PINN_SIREN	0.21	0.0068	21 057	5.85×10^{-6}
MSF-PINN	0.43	0.0142	2 110 049	9.99×10^{-5}
PINN	0.71	0.0232	21 057	6.72×10^{-5}

A continuación se presenta una breve introducción para la imagen que muestra el rendimiento de los modelos en función del error relativo y la complejidad (número de parámetros):

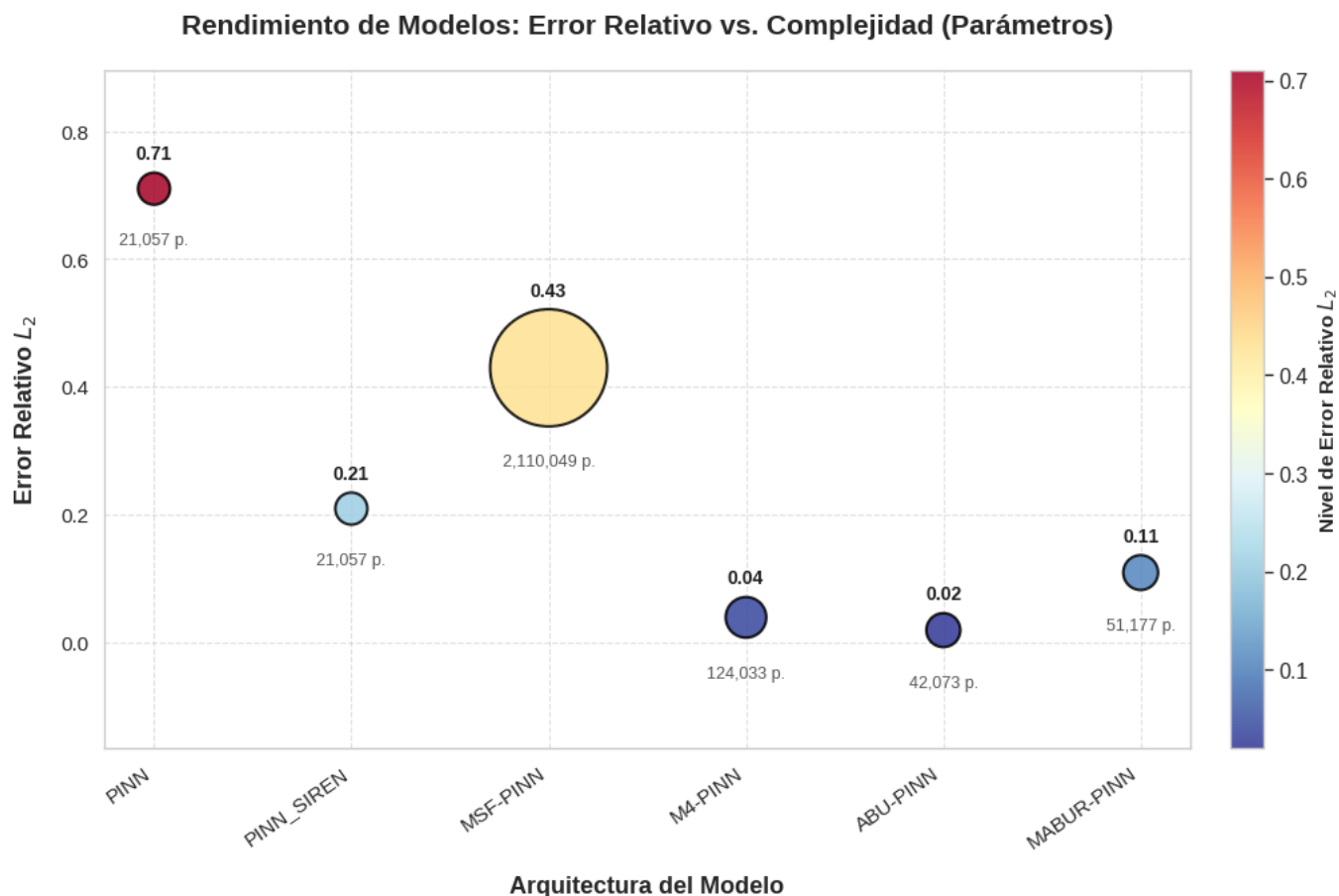


Figura 3: Diagrama comparativo de error relativo vs Complejidad (parámetros) para cada arquitectura.

6.1. Comportamiento del optimizador ADAM + LRA

Todas las arquitecturas fueron entrenadas inicialmente con ADAM y *Learning Rate Annealing* (LRA) para equilibrar los términos de la pérdida compuesta. Los resultados finales de pérdida (costo) muestran que:

- ABU-PINN alcanza la pérdida más baja (4.60×10^{-7}), seguido de M4-PINN (2.21×10^{-6}) y PINN_SIREN (5.85×10^{-6}).
- La PINN vanilla y MSF-PINN presentan pérdidas más altas (del orden de 10^{-5} y 10^{-4} respectivamente), indicando dificultades para minimizar simultáneamente los residuos de la PDE y las condiciones de contorno.

Las curvas de pérdida muestran que:

- ADAM + LRA reduce la pérdida total a órdenes de 10^{-5} tras 10^5 épocas, pero con ruido debido al reajuste periódico de los pesos λ_i .
- En ABU-PINN y M4-PINN las curvas son más estables y la pérdida desciende rápidamente, indicando un mejor balance de gradientes.
- MSF-PINN muestra alta varianza, probablemente por su elevado número de parámetros y la dificultad de entrenar embeddings multi-escala.

Un aspecto clave es que la activación sinusoidal (SIREN) favorece el aprendizaje de condiciones de contorno periódicas, alcanzando pérdidas en BC del orden 10^{-12} . Sin embargo, su error relativo final (0.21) es superior al de ABU-PINN y M4-PINN, lo que indica que una pérdida muy baja no siempre se traduce en menor error de generalización.

7. Conclusiones y recomendaciones

1. **ABU-PINN** es la arquitectura más equilibrada: menor error con costo computacional moderado y pérdida final muy baja. Su capacidad para aprender la combinación óptima de funciones de activación (incluyendo sinusoides y exponenciales) resulta clave para problemas con componentes periódicas y de rápida variación.

2. **M4-PINN** es competitivo, pero su mayor número de parámetros no justifica la pequeña ganancia en error respecto a ABU-PINN.
3. **MSF-PINN** no es recomendable para este tipo de problema debido a su alto costo y pobre rendimiento. El uso de múltiples escalas de Fourier requiere un ajuste muy fino de σ y una arquitectura que evite el sobreajuste.
4. **PINN_SIREN** mejora notablemente a la PINN vanilla gracias a su activación periódica, pero se queda lejos de ABU-PINN. Puede ser una buena opción si se prioriza la simplicidad.
5. Para trabajos futuros, se recomienda explorar **ABU-PINN con conexiones residuales adaptativas** pero con funciones de activación adecuadas y arquitectura que no complejice la red o implementar un mecanismo de *gate* más sofisticado que evite interferencias entre las dos fuentes de adaptabilidad.

8. Recursos y declaraciones

8.1. Recursos

Todos los recursos se encuentran en el siguiente github:
<https://github.com/Emmaquantum/HPINN>

8.2. Declaración sobre el uso de Inteligencia Artificial

Se utilizaron herramientas de IA para la búsqueda y recopilación de bibliografía, así como para la revisión y corrección de código y la optimización de la redacción de este documento.

9. Anexos

9.1. Vanilla PINN

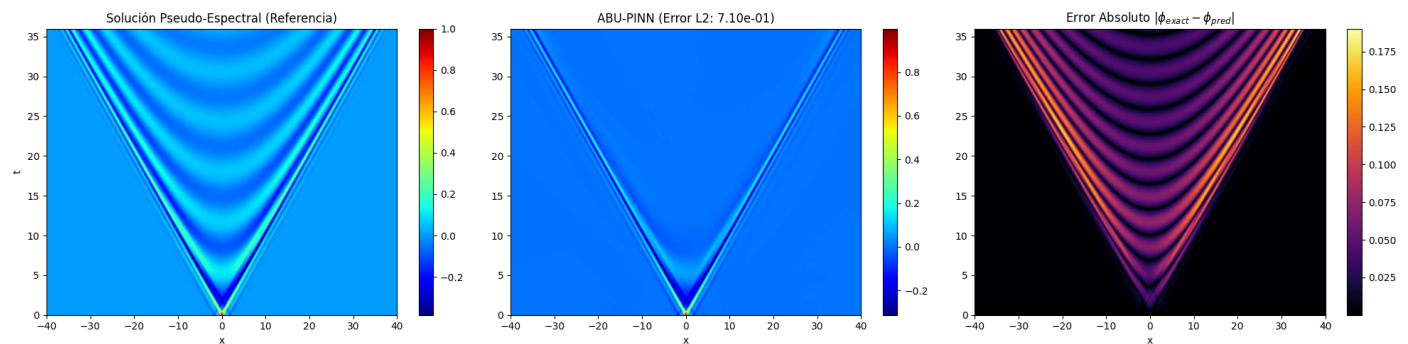


Figura 4: Error puntual en el dominio espacial para la PINN vanilla. Se observan errores elevados (máximos superiores a 0.02) distribuidos en toda la región, indicando dificultades para aproximar la solución.

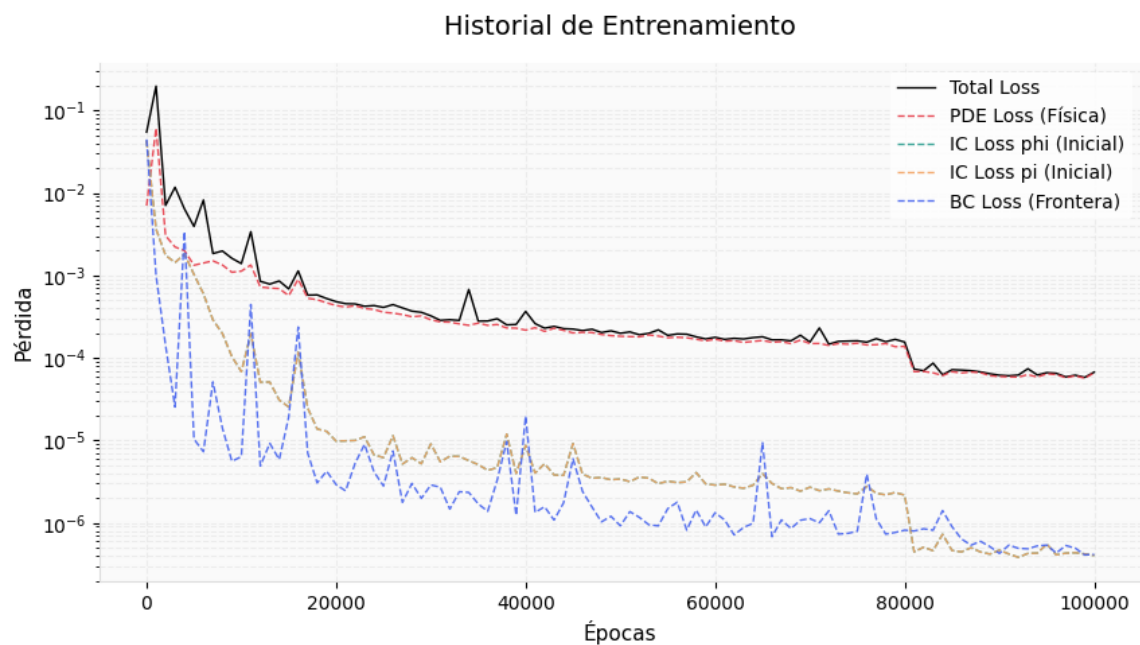


Figura 5: Evolución de las funciones de pérdida durante el entrenamiento de la PINN vanilla. La pérdida total presenta oscilaciones y una convergencia lenta, con valores finales del orden de 10^{-5} .

9.2. PINN - SIREN

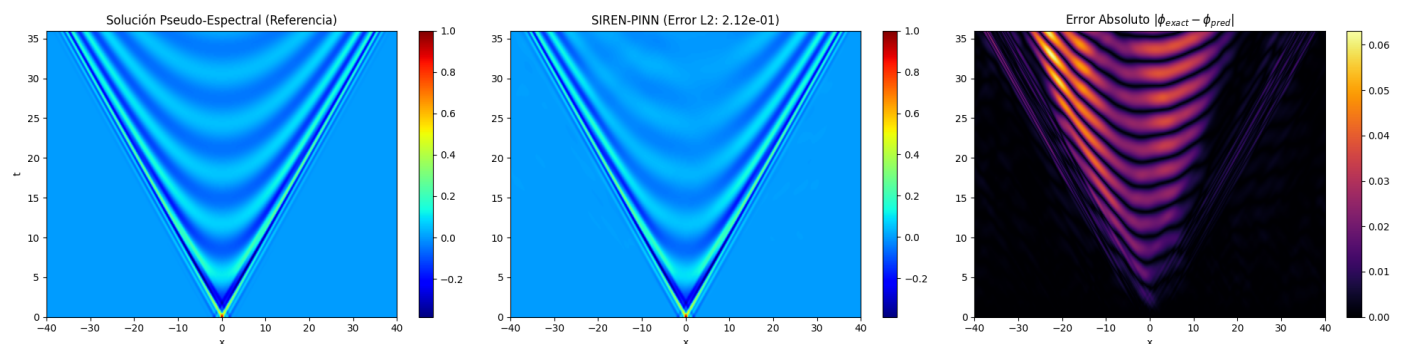


Figura 6: Mapa de error espacial de la PINN con activación SIREN. Los errores máximos se reducen respecto a la PINN vanilla (aproximadamente 0.0068), con una mejor captura de las oscilaciones.

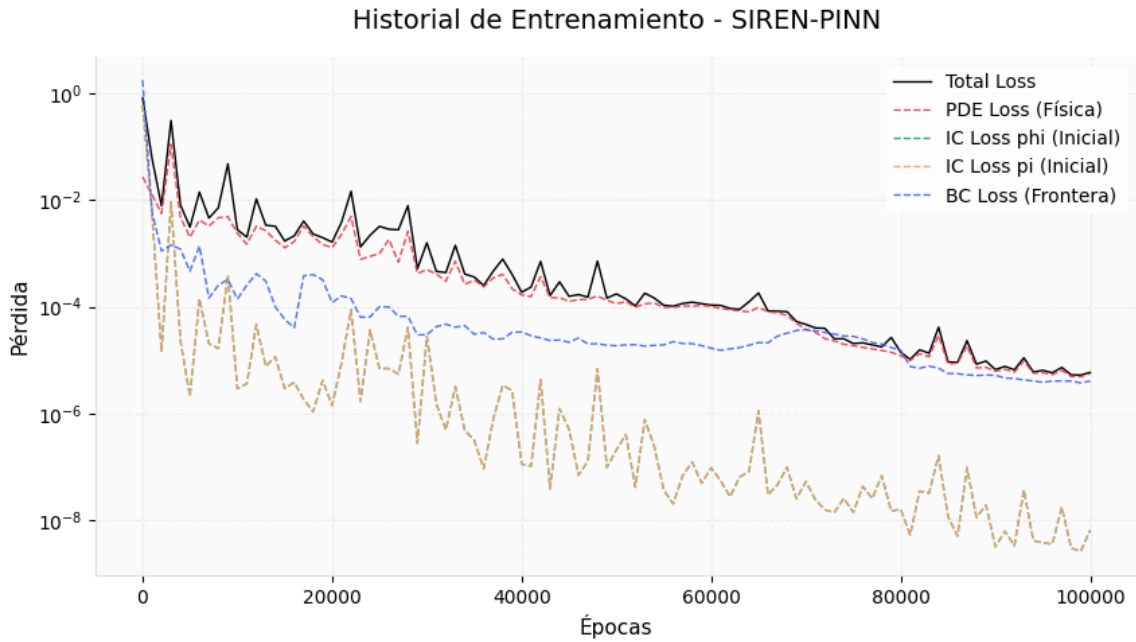


Figura 7: Historial de pérdidas para la PINN-SIREN. La pérdida asociada a las condiciones de contorno alcanza valores muy bajos ($\sim 10^{-12}$), mientras que la pérdida total converge a $\sim 5.85 \times 10^{-6}$.

9.3. MultiScaleFourier PINN

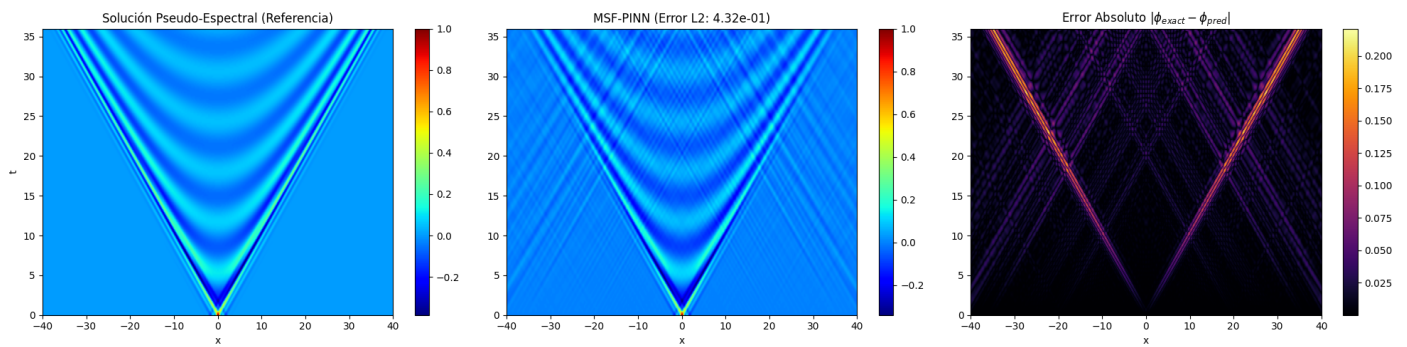


Figura 8: Error puntual de la arquitectura MSF-PINN. A pesar del alto número de parámetros, los errores son significativos (máximos ~ 0.0142), especialmente en las regiones donde la solución presenta altas frecuencias.

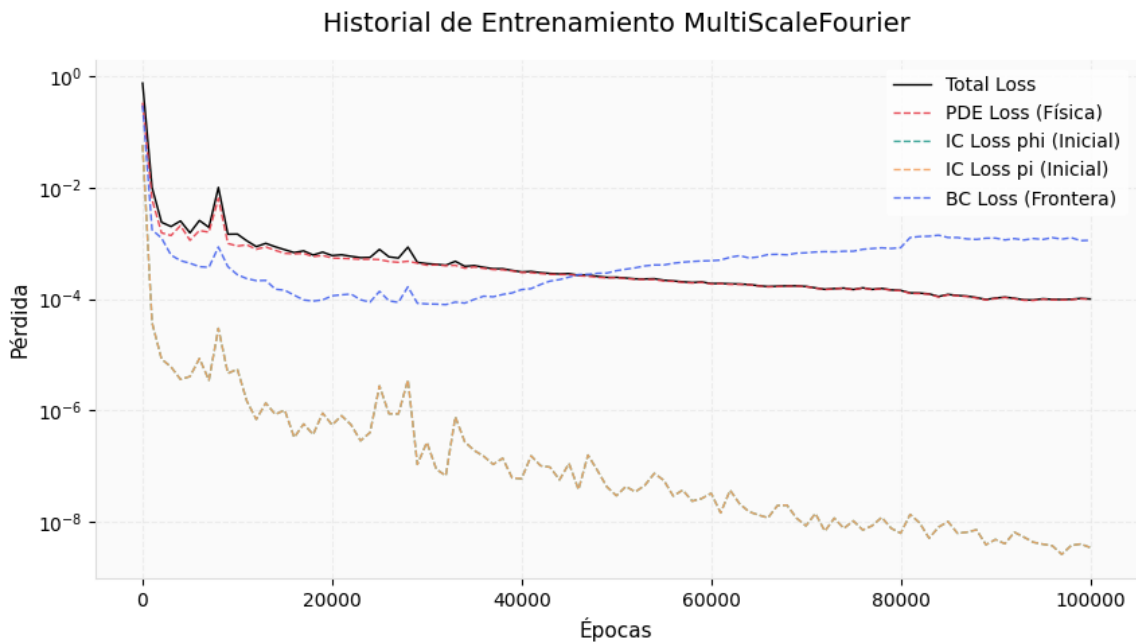


Figura 9: Curvas de pérdida durante el entrenamiento de MSF-PINN. Se aprecia una alta varianza y una convergencia pobre, con pérdida total final del orden de 10^{-4} .

9.4. Modified MLP PINN

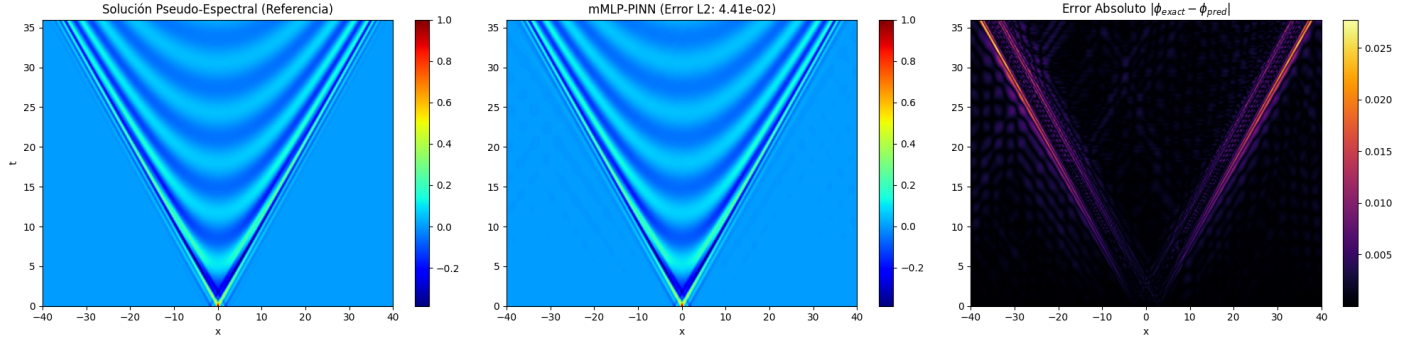


Figura 10: Distribución del error espacial para la arquitectura M4-PINN. Los errores máximos se reducen drásticamente (~ 0.0013), con una distribución más homogénea.

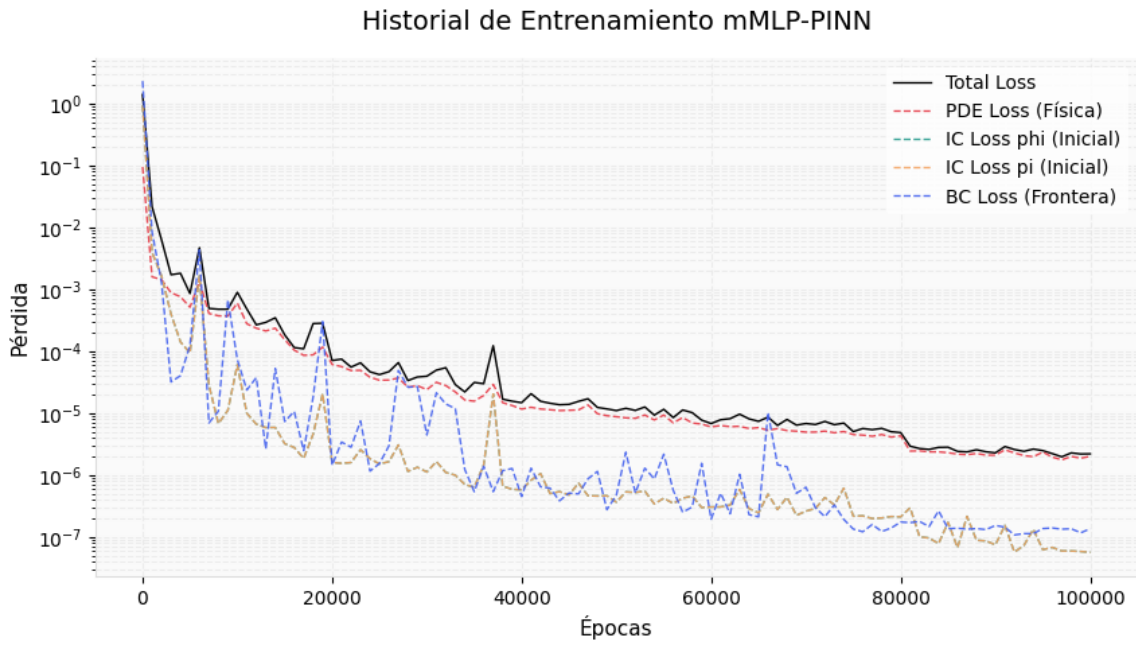


Figura 11: Evolución de las funciones de pérdida en M4-PINN. La pérdida total desciende rápidamente y se estabiliza en $\sim 2.21 \times 10^{-6}$, reflejando un mejor balance de gradientes.

9.5. AbuPINN

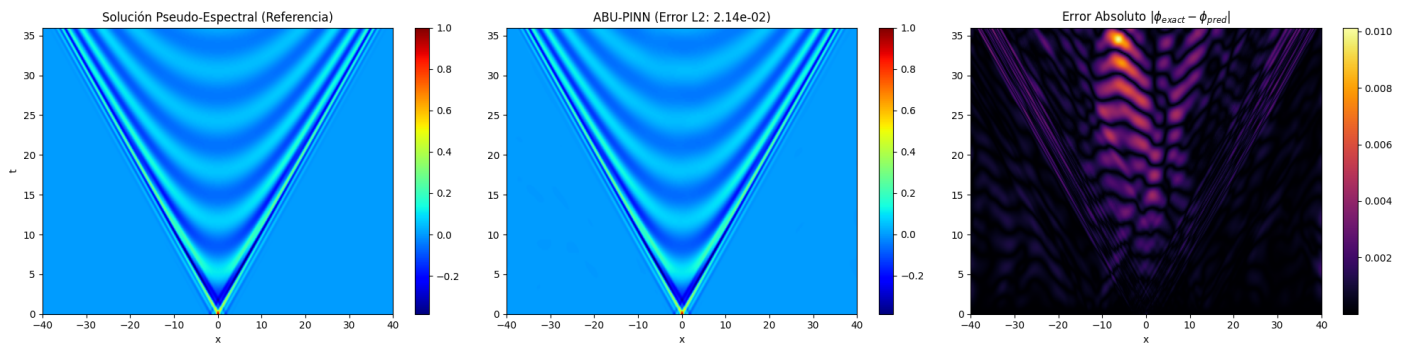


Figura 12: Mapa de error espacial de ABU-PINN. Presenta el error máximo más bajo (0.0008) entre todas las arquitecturas, con una distribución homogénea y sin picos pronunciados.

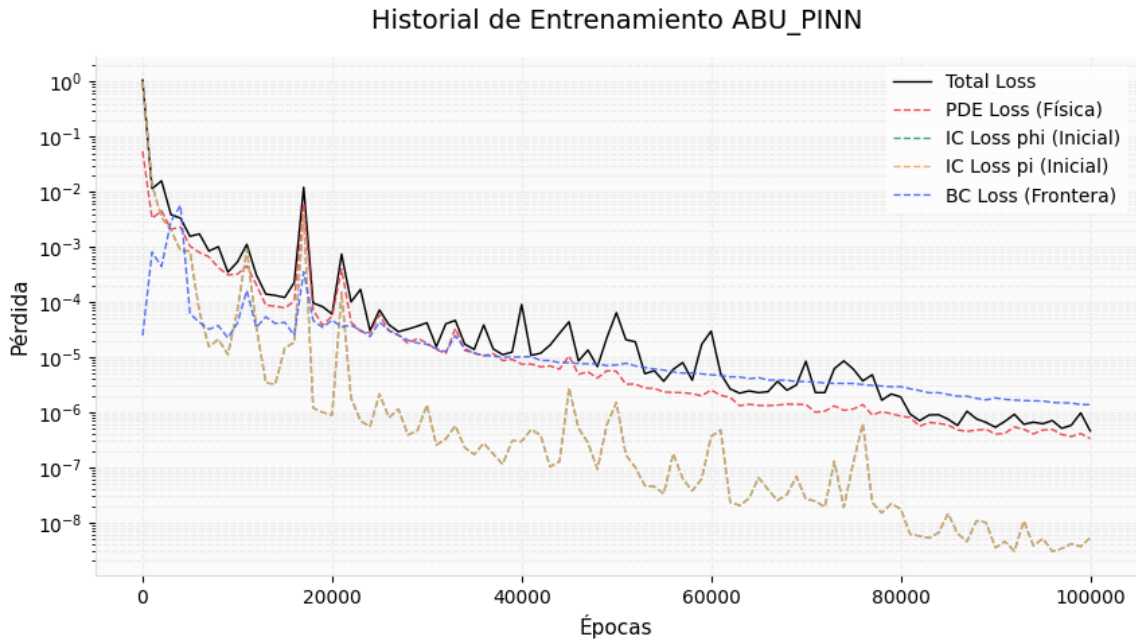


Figura 13: Historial de pérdidas de ABU-PINN. La pérdida total alcanza el valor más bajo (4.60×10^{-7}) con una convergencia estable y rápida, indicando una excelente adaptabilidad de la función de activación.

9.6. MaburPINN

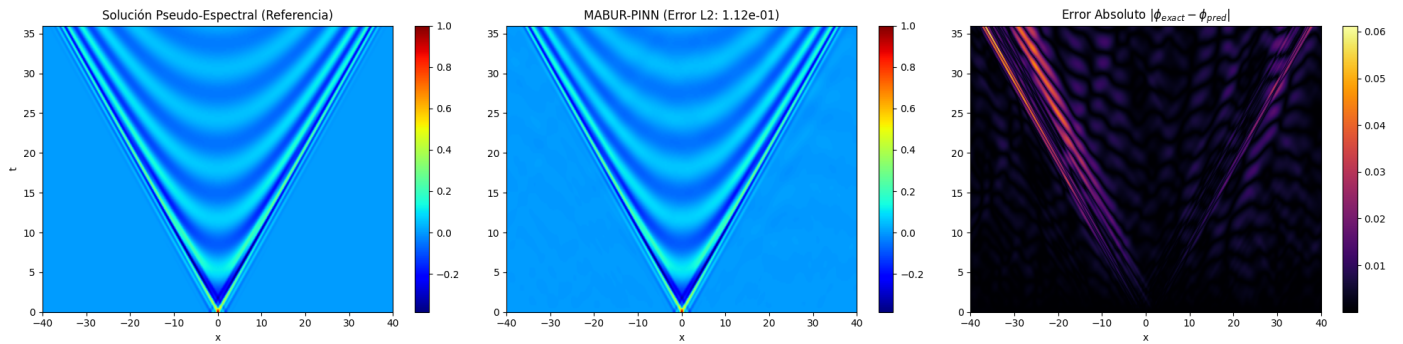


Figura 14: Error espacial de la arquitectura propuesta MABUR-PINN. Aunque el error máximo (0.0037) es superior al de ABU-PINN, se observan mejoras en regiones de alta frecuencia respecto a MSF-PINN.

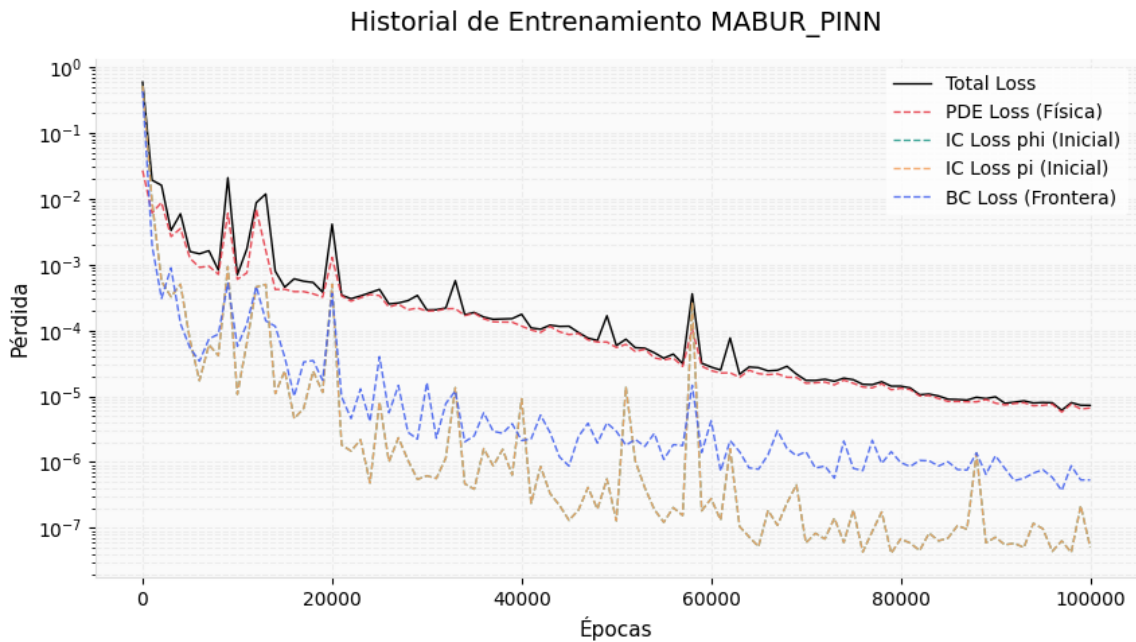


Figura 15: Curvas de pérdida durante el entrenamiento de MABUR-PINN. La pérdida total converge a $\sim 7.25 \times 10^{-6}$, mostrando una estabilidad intermedia entre M4-PINN y AbuPINN.

10. Referencias

Referencias

- [1] Amari, S. (1998). Natural gradient works efficiently in learning. *Neural Computation*, 10(2), 251–276.
- [2] Boullé, N. (2020). Rational neural networks. *arXiv preprint arXiv:2004.01993*.
- [3] Boyd, S., & Vandenberghe, L. (2004). *Convex optimization*. Cambridge University Press.
- [4] Chrisnanto, J. E., Alia, S. R., Fadillah, N., & Chrisnanto, Y. H. (2026). High-fidelity modeling of stochastic chemical dynamics on complex manifolds: A multi-scale SIREN-PINN framework for the curvature-perturbed Ginzburg-Landau equation. *arXiv preprint arXiv:2601.08104*.
- [5] Dubey, S. R., Singh, S. K., & Chaudhuri, B. B. (2022). Activation functions in deep learning: A comprehensive survey and benchmark. *Neurocomputing*, 503, 92–108.
- [6] Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- [7] Krishnapriyan, A. S., Gholami, A., Zhe, S., Kirby, R. M., & Mahoney, M. W. (2021). Characterizing possible failure modes in physics-informed neural networks. *Advances in Neural Information Processing Systems*, 34, 26548–26560.
- [8] Madani, Y. A., Mohamed, K. S., Yasin, S., Ramzan, S., Aldwoah, K., & Hassan, M. (2025). Exploring novel solitary wave phenomena in Klein–Gordon equation using ϕ^6 model expansion method. *Scientific Reports*, 15(1), 1834.
- [9] Megías, E., Teixeira, M. J., Timoteo, V. S., & Deppman, A. (2022). Nonlinear Klein–Gordon equation and the Bose–Einstein condensation. *The European Physical Journal Plus*, 137(3), 325.
- [10] Miller, P. D., Perry, P. A., Saut, J. C., & Sulem, C. (Eds.). (2019). *Nonlinear dispersive partial differential equations and inverse scattering*. Springer.
- [11] Pascanu, R., & Bengio, Y. (2013). Revisiting natural gradient for deep networks. *arXiv preprint arXiv:1301.3584*.
- [12] Qian, Y., Zhang, Y., Huang, Y., & Dong, S. (2023). Physics-informed neural networks for approximating dynamic (hyperbolic) PDEs of second order in time: Error analysis and algorithms. *Journal of Computational Physics*, 495, 112527.
- [13] Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378, 686–707.
- [14] Rohrhofer, F. M., Geist, A. R., Aichhorn, M., & von der Linden, W. (2023). Pareto-frontier analysis for multi-objective optimization in physics-informed neural networks. *arXiv preprint arXiv:2302.08765*.
- [15] Ruder, S. (2016). An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*.
- [16] Shin, Y., Darbon, J., & Karniadakis, G. E. (2020). On the convergence of physics informed neural networks for linear second-order elliptic and parabolic type PDEs. *arXiv preprint arXiv:2004.01806*.
- [17] Sitzmann, V., Martel, J., Bergman, A., Lindell, D., & Wetzstein, G. (2020). Implicit neural representations with periodic activation functions. *Advances in Neural Information Processing Systems*, 33, 7462–7473.
- [18] Tancik, M., Srinivasan, P. P., Mildenhall, B., Fridovich-Keil, S., Raghavan, N., Singhal, U., Ramamoorthi, R., Barron, J. T., & Ng, R. (2020). Fourier features let networks learn high frequency functions in low dimensional domains. *arXiv preprint arXiv:2006.10739*.
- [19] Trefethen, L. N., & Bau, D. (1997). *Numerical linear algebra*. SIAM.
- [20] Wang, S., Li, B., Chen, Y., & Perdikaris, P. (2024). PirateNets: Physics-informed deep learning with residual adaptive networks. *arXiv preprint arXiv:2402.00326*.

- [21] Wang, S., Lu, L., Song, S., & Huang, G. (2023). Learning specialized activation functions for physics-informed neural networks. *arXiv preprint arXiv:2308.04073*.
- [22] Wang, S., Teng, Y., & Perdikaris, P. (2021b). Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5), A3055–A3081.
- [23] Wang, S., Wang, H., & Perdikaris, P. (2021a). On the eigenvector bias of fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 384, 113938.
- [24] Wang, S., Yu, X., & Perdikaris, P. (2022). When and why PINNs fail to train: A neural tangent kernel perspective. *Journal of Computational Physics*, 449, 110768.
- [25] Xu, Z., Zhao, Y., Li, Y., & Zhang, H. (2025). Input normalization for physics-informed neural networks: A comparative study. *Journal of Machine Learning for Modeling and Computing*, 6(1), 45–62.
- [26] Yagdjian, K. (2013). Semilinear hyperbolic equations in curved spacetime. In *Fourier Analysis: Pseudo-differential Operators, Time-Frequency Analysis and Partial Differential Equations* (pp. 391–415). Springer.